[17] Intro

PyTorch and Neural Networks

Intro to PyTorch and Neural Networks

Welcome

Welcome! In this lesson, you'll learn about neural networks using the `PyTorch` library. Starting from scratch, you'll learn

- how the neural network training process works
- the role of
  activation functions
  Preview: Docs Activation functions are mathematical functions that introduce non-linearity into the model, enabling neural networks to learn complex patterns from data.
  ,
  loss functions
  Preview: Docs Represents critical functions used to optimize neural network predictions in PyTorch
  , and optimization algorithms
- the concepts behind loss functions like Mean Squared Error and optimization algorithms like Gradient Descent
- how to evaluate a neural network's performance
- how to build neural networks for regression problems

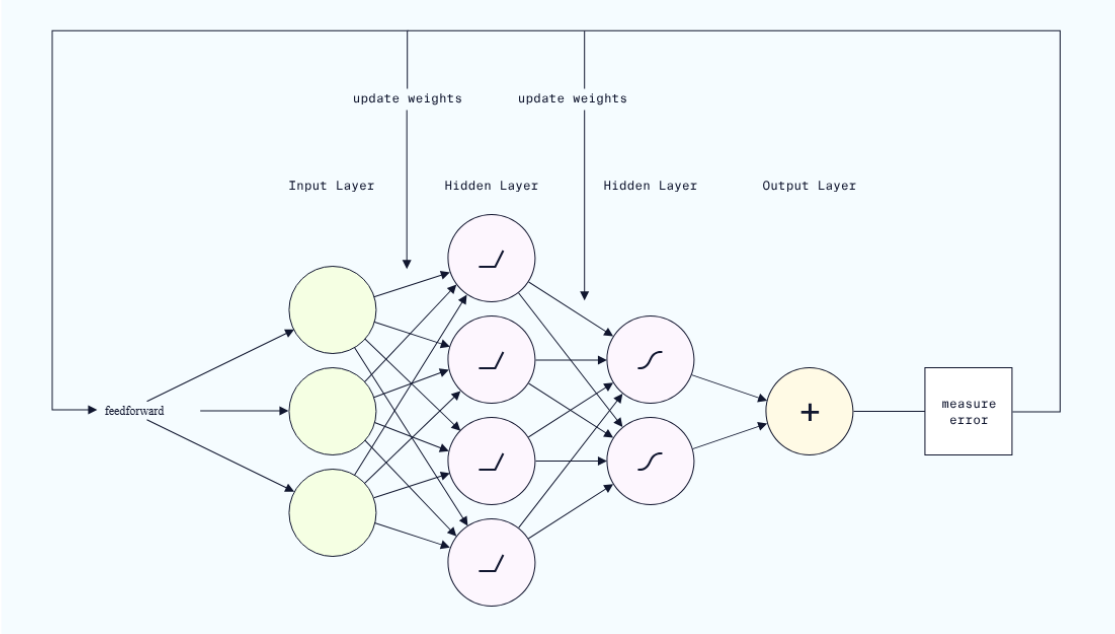
Along the way, you'll use `PyTorch` to build, train, and test your own neural network for predicting rent prices from a StreetEasy dataset.

Who is this course for?

This course assumes no prior experience with neural networks. We also cover the algorithms conceptually, so there is no need to understand calculus or linear algebra before taking this course. However, we do recommend having a basic working knowledge of Python, NumPy, and pandas for data science before taking this course.

We will review the basics of Machine Learning (ML) as we go, but we recommend having a basic understanding of Machine Learning concepts (like supervised/unsupervised learning and train/test splits) to get the most out of this course.

We're so excited to see what you build!



PyTorch Activation Functions

The Activation Functions in PyTorch are a collection of pre-built functions essential for constructing neural networks. These mathematical functions determine the output of each neuron by assessing whether its input is relevant for the model's prediction, effectively deciding whether the neuron should be activated. Activation functions also help normalize neuron outputs, typically constraining them to a range between 0 and 1 or -1 and 1.

By introducing non-linearity into the network, activation functions enable the model to learn complex patterns in the data. Common activation functions include `ReLU`, `ReLU6`, `Leaky ReLU`, `Sigmoid`, `Tanh`, and `Softmax`, which are applied to the outputs of neurons throughout the network.

ReLU

ReLU (Rectified Linear Unit) is a popular activation function that returns the input if it is positive, and zero otherwise. It is mathematically defined as:

```
f(x) = max(0, x)
```

ReLU is widely used in deep learning models due to its simplicity and efficiency. It enables the model to learn complex patterns in the data by introducing non-linearity.

```
import torch.nn as nn
# Define a neural network model with ReLU activation function
class SimpleNN(nn.Module):
 def __init__(self):
   super(SimpleNN, self).__init__()
   self.fc1 = nn.Linear(10, 5)
   self.fc2 = nn.Linear(5, 1)
   self.relu = nn.ReLU()
 def forward(self, x):
   x = self.relu(self.fc1(x))
   x = self.fc2(x)
   return x
# Create a model instance
model = SimpleNN()
```

Copy to clipboard

ReLU6

ReLU6 is a variation of the ReLU activation function that returns the input if it is positive and less than or equal to 6, and returns `0` if the input is negative or greater than 6. It is mathematically defined as:

```
f(x) = min(max(0, x), 6)
```

ReLU6 is useful when you want to limit the output of the activation function to a specific range, such as between `0` and 6.

```
import torch.nn as nn
# Define a neural network model with ReLU6 activation function
class SimpleNN(nn.Module):
 def __init__(self):
   super(SimpleNN, self).__init__()
   self.fc1 = nn.Linear(10, 5)
   self.fc2 = nn.Linear(5, 1)
   self.relu6 = nn.ReLU6()

 def forward(self, x):
   x = self.relu6(self.fc1(x))
   x = self.fc2(x)
   return x
# Create a model instance
model = SimpleNN()
```

Copy to clipboard

Leaky ReLU

Leaky ReLU is a variation of the ReLU activation function that returns the input if it is positive, and a small fraction of the input otherwise. It is mathematically defined as:

```
f(x) = max(0.01x, x)
```

Leaky ReLU helps prevent the dying ReLU problem, where neurons can become inactive and stop learning. By allowing a small gradient for negative inputs, Leaky ReLU ensures that all neurons in the network continue to learn, promoting better training and performance.

```
import torch.nn as nn
# Define a neural network model with Leaky ReLU activation function
class SimpleNN(nn.Module):
 def __init__(self):
   super(SimpleNN, self).__init__()
   self.fc1 = nn.Linear(10, 5)
   self.fc2 = nn.Linear(5, 1)
   self.leaky_relu = nn.LeakyReLU()

 def forward(self, x):
   x = self.leaky_relu(self.fc1(x))
   x = self.fc2(x)
   return x

# Create a model instance
model = SimpleNN()
```

Copy to clipboard

Sigmoid

Sigmoid is a popular activation function that returns a value between `0` and `1`. It is mathematically defined as:

```
f(x) = 1 / (1 + e^(-x))
```

The Sigmoid function is commonly used in binary classification problems where the output needs to be constrained between `0` and `1`. It is also frequently utilized in the output layer of neural networks to predict probabilities, making it suitable for tasks that require a binary outcome.

```
import torch.nn as nn
# Define a neural network model with Sigmoid activation function
class SimpleNN(nn.Module):
 def __init__(self):
   super(SimpleNN, self).__init__()
   self.fc1 = nn.Linear(10, 5)
   self.fc2 = nn.Linear(5, 1)
   self.sigmoid = nn.Sigmoid()

 def forward(self, x):
   x = self.sigmoid(self.fc1(x))
   x = self.fc2(x)
   return x

# Create a model instance
model = SimpleNN()
```

Copy to clipboard

Tanh

Tanh (hyperbolic tangent) is an activation function that returns a value between `-1` and `1`. It is mathematically defined as:

```
f(x) = (e^x - e^(-x)) / (e^x + e^(-x))
```

Tanh is similar to the Sigmoid function but differs in that it outputs values within the range of `-1` to `1`. This property makes it particularly useful in the hidden layers of neural networks, as it helps introduce non-linearity and can lead to improved convergence during training.

```
import torch.nn as nn
# Define a neural network model with Tanh activation function
class SimpleNN(nn.Module):
 def __init__(self):
   super(SimpleNN, self).__init__()
   self.fc1 = nn.Linear(10, 5)
   self.fc2 = nn.Linear(5, 1)
   self.tanh = nn.Tanh()
 def forward(self, x):
   x = self.tanh(self.fc1(x))
   x = self.fc2(x)
   return x
# Create a model instance
model = SimpleNN()
```

Copy to clipboard

Softmax

Softmax is an activation function that returns a probability distribution over multiple classes. It is mathematically defined as:

```
f(x_i) = e^(x_i) / sum(e^(x_j)) for j = 1 to n
```

Softmax is commonly used in the output layer of neural networks for multi-class classification problems. It ensures that the output values sum to 1, allowing the model's predictions to be interpreted as probabilities for each class. This property makes it particularly useful for tasks with multiple classes, providing a clear way to identify the predicted class.

```
import torch.nn as nn
# Define a neural network model with Softmax activation function
class SimpleNN(nn.Module):
 def __init__(self):
   super(SimpleNN, self).__init__()
   self.fc1 = nn.Linear(10, 5)
   self.fc2 = nn.Linear(5, 3)
   self.softmax = nn.Softmax(dim=1)
 def forward(self, x):
   x = self.fc1(x)
   x = self.fc2(x)
   x = self.softmax(x)
   return x
# Create a model instance
model = SimpleNN()
```

Intro to PyTorch and Neural Networks

Intro to Tensors

Tensors are the fundamental building blocks of neural networks in PyTorch. Similar to NumPy arrays,
tensors
Preview: Docs A tensor is a multi-dimensional array of values of the same type.
act as storage containers for numerical data.
Most of the time, our data starts out in NumPy arrays or pandas DataFrames. In order to use the data in PyTorch, we have to convert these datatypes to tensors using PyTorch's `torch.tensor()` method.
The `torch.tensor()` method takes two arguments:

- the numerical data (NumPy array, Python list, or Python numeric variable)
- the desired data type (the `dtype` parameter)

Common datatypes include `torch.int` for integers and `torch.float` for floats.
Throughout this lesson, we'll be trying to build a neural network to predict the rent of an apartment. The dataset we'll be using was produced by StreetEasy for our original Intro to Machine Learning course.
Here's an example of how we could convert a single apartment's rent to an integer tensor:

```
# define the apartment's rent
```

```
rent = 2550
# convert to an integer tensor
rent_tensor = torch.tensor(
    rent,
    dtype=torch.int)
```

Copy to Clipboard

Now, let's create a tensor that includes more information about our apartment: `rent`, `size_sqft`, and `age`.

```
# create a numpy array with the rent in US dollars, size in square
feet, and age in years
apt_array = np.array([2550, 750, 3.5])
# convert to a tensor of floats
apt_tensor = torch.tensor(
    apt_array,
    dtype=torch.float)
```

Copy to Clipboard

Often, our data will start out as a pandas DataFrame. In this case, we need to use the `.values` attribute of the DataFrame to retrieve the data as a NumPy array:

```
# convert a DataFrame named df to a PyTorch tensor
torch.tensor(
    df.values,
    dtype=torch.float)
```

Copy to Clipboard

Note that working with individuals columns of a DataFrame can occasionally cause problems due to dimension assumptions in torch.

One solution is to make sure the individual column is actually a full DataFrame by using two brackets to select it:

```
torch.tensor(
    df[['column']].values,
    dtype=torch.float)
```

Copy to Clipboard

Another is to use torch's `.view(-1,1)` method to automatically adjust dimensions:

```
torch.tensor(
    df['column'].values,
    dtype=torch.float).view(-1,1)
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Before working with tensors, let's practice running Jupyter Notebook cells and testing code on Codecademy. We've loaded a Jupyter Notebook in the workspace for this exercise.

Each Jupyter Notebook consists of **text cells** containing instructions and **code cells** for writing Python code.
This checkpoint (Checkpoint 1) has a code cell that imports PyTorch, NumPy, and pandas.
To complete this checkpoint, perform the following:

- Select the code cell containing the three import statements (click anywhere in the cell)
- Click the `Run` button or the `Shift`+`Enter/Return` keys (see image below)
- Click the `Save` button or use the `Control/Command`+`S` keys to save your work

- Click the `Test Work` button below the Jupyter Notebook to check if you've completed the Checkpoint!

If you've successfully completed the Checkpoint, you'll get a checkmark at the top of these Checkpoint 1 instructions, and be able to continue on to the next checkpoint.
When all checkpoints are complete, the `Next` button at the bottom right will become clickable.
Checkpoint 2 Passed

2.

Create a tensor containing the values `[2000,500, 7]` with the `torch.int` data type.
Assign your tensor to the variable `apartment_tensor`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
The syntax for `torch.tensor` is

```
torch.tensor(numerical_data, dtype = desired_datatype)
```

Copy to Clipboard
Checkpoint 3 Enabled

3.

In the Checkpoint 3 code cell, we've already added code to create a DataFrame named `apartments_df`. This array includes data on rent, size, and age of the apartments in our dataset.
Use `torch.tensor()` to convert this DataFrame to a tensor, with `torch.float32` as the datatype. Assign the result to the variable `apartments_tensor`.
Add your solution to the code cell directly below the comment `## YOUR SOLUTION HERE ##`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
When working with DataFrames, use the `.values` attribute to access the numerical data as a NumPy array before converting to a tensor.

Follow the Checkpoint 1 instructions (below the exercise text to the left) to run the following code cell, save the notebook, and run our code tests.

```
# do not modify this code
import pandas as pd
import torch
import numpy as np
```

Checkpoint 2/3

Create a tensor containing the values `[2000,500, 7]` with the `torch.int` data type. Assign your tensor to the variable `apartment_tensor`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##

apartment_tensor = torch.tensor(
   np.array([2000,500, 7]),
   dtype=torch.int
)

# show output
apartment_tensor
```

```
tensor([2000,  500,    7], dtype=torch.int32)
```

Checkpoint 3/3

In this checkpoint's code cell, we've already added code to create a DataFrame named `apartments_df`. This array includes data on rent, size, and age of the apartments in our dataset.
Use `torch.tensor()` to convert this DataFrame to a tensor, with `torch.float32` as the datatype. Assign the result to the variable `apartments_tensor`.
Add your solution to the code cell directly below the comment `## YOUR SOLUTION HERE ##`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# import the dataset using pandas
apartments_df = pd.read_csv("streeteasy.csv")

# select the rent, size, and age columns
apartments_df = apartments_df[["rent", "size_sqft", "building_age_yrs"]]

## YOUR SOLUTION HERE ##
apartments_tensor = torch.tensor(
   apartments_df.values,
   dtype = torch.float)

# show output
apartments_tensor
```

```
tensor([[2.5500e+03, 4.8000e+02, 1.7000e+01],
        [1.1500e+04, 2.0000e+03, 9.6000e+01],
        [3.0000e+03, 1.0000e+03, 1.0600e+02],
        ...,
        [1.6990e+03, 2.5000e+02, 9.6000e+01],
        [3.4750e+03, 6.5100e+02, 1.4000e+01],
        [4.5000e+03, 8.1600e+02, 9.0000e+00]])
```

Intro to PyTorch and Neural Networks

Linear Regression Review

During this lesson, we'll be creating a neural network to predict the *rent* of an apartment based on features like *square footage*.
This is an example of a **regression problem** in machine learning: a problem where we are trying to predict a target numeric value.
In any regression model, we start with certain **input features**. Using those input features, we try to predict the **target** or **output** (sometimes called a **label**, although this is more common in classification problems).
For example, we might try to build a model with

- **input feature**: square footage of an apartment
- **target**: predicted cost to rent the apartment

Linear Regression

The most basic kind of regression is linear regression. If you've never encountered linear regression before, we will do a quick review in this exercise, but it may be worth checking out our Intro to Machine Learning course.

Linear regression models use **linear equations** to make predictions. The most basic linear equation is the standard equation of a line:

$$y=mx+b$$

$$y=mx+b$$

For example, here's a linear equation that uses square footage to predict rent:

$$rent=2.5sz\_sqft+1000$$

$$rent=2.5sz\_sqft+1000$$

This model claims that if we

- take the square footage of an apartment
- multiply it by 2.5, and
- add 1000

we will get the apartment rent as output.

For example, if we're looking at an apartment with 500 square feet, we would predict

$$rent=2.5\times500+1000=2250$$

$$rent=2.5\times500+1000=2250$$

When talking about linear equations, we tend to use words like "slope" and "intercept". In the world of machine learning, and particularly neural networks, we'll use slightly different language:

- `rent` is the **output** or **target**
- `sz_sqft` is the **input feature**
- `2.5` is the **weight** associated to `sz_sqft`
- `1000` is the **bias**

Models will often have more than one input feature. For example, we might be able to improve a model of rent costs by including the age of the apartment building.

To include age in our model, we just need to add `age` as an input feature together with its own weight.

Here's a concrete example:

$$rent=2.5sqft-1.5age+1000$$

$$rent=2.5sqft-1.5age+1000$$

Note that the weight associated to `age` is `-1.5`, with the negative sign included.

In general, any linear model consists of:

- some number of input features
- each input feature is multiplied by its own weight, creating a **weighted input feature**
- all the weighted input features are added together with the bias

While the neural network models we will build in this course go beyond linear regression, many of these ideas (like inputs, weights, and biases) are present in even the most advanced neural network models!

Instructions

Checkpoint 1 Passed

1.

Suppose we are modeling the cost of rent using the linear equation

$$rent=3sz\_sqft+500$$

$$rent=3sz\_sqft+500$$

Use the code cell in this checkpoint to calculate the predicted rent for a 500 square foot apartment. Assign your answer to the variable `predicted_rent`.
Don't forget to run and save the cell before selecting `Test Work`!
Substitute `500` for the input feature in the equation.
Checkpoint 2 Passed

2.

Suppose we are modeling the cost of rent using the multiple linear equation
rent=3sz_sqft+10bedrooms+250

rent=3sz_sqft+10bedrooms+250

What is the weight associated with the input feature `bedrooms`? Assign your answer to the variable `bedroom_weight`.
Don't forget to run and save the cell before selecting `Test Work`!
An input feature is always multiplied by its weight in the linear equation.
Checkpoint 3 Passed

3.

Suppose we are modeling the cost of rent using the multiple linear equation
rent=3sz_sqft+10bedrooms+250

rent=3sz_sqft+10bedrooms+250

What is the bias in this model? Assign your answer to the variable `bias`.
Don't forget to run and save the cell before selecting `Test Work`!
The bias is added to the linear equation after the weighted input features are added together.

Suppose we are modeling the cost of rent using the linear equation

rent=3sz_sqft+500

Use the code cell below to calculate the predicted rent for a 500 square foot apartment. Assign your answer to the variable `predicted_rent`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Forgetting to save is often the cause of mystery errors. See the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
predicted_rent = 3*500 + 500
```

Checkpoint 2/3

Suppose we are modeling the cost of rent using the multiple linear equation

rent=3sz_sqft+10bedrooms+250

What is the weight associated with the input feature `bedrooms`? Assign your answer to the variable `bedroom_weight`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Forgetting to save is often the cause of mystery errors. See the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
bedroom_weight = 10
```

Checkpoint 3/3

Suppose we are modeling the cost of rent using the multiple linear equation

rent=3sz_sqft+10bedrooms+250

What is the bias in this model? Assign your answer to the variable `bias`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Forgetting to save is often the cause of mystery errors. See the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
bias = 250
```

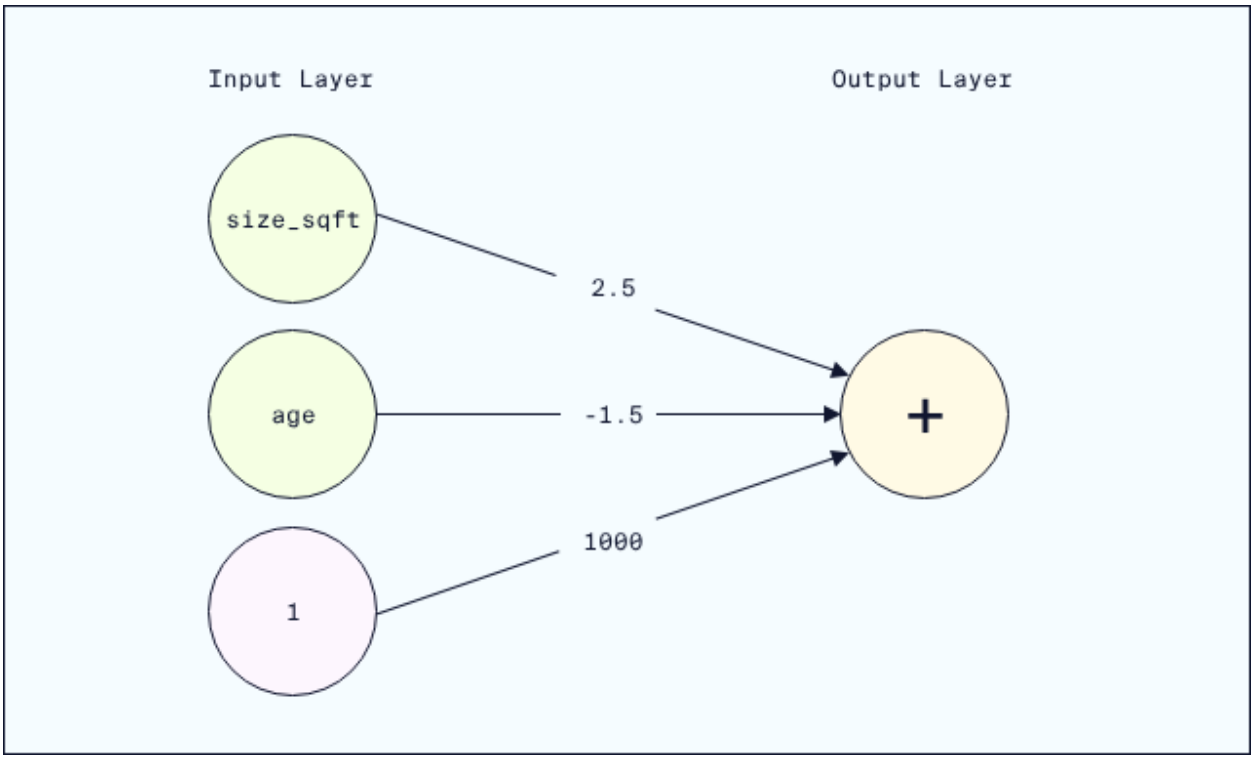
Linear Regression with Perceptrons

In the previous exercise, we used the following linear equation to predict rent costs based on square footage and building age:

rent=2.5sqft−1.5age+1000

$$rent = 2.5sqft - 1.5age + 1000$$

Our first step toward building neural networks is to transform this equation into a neural network structure called a **Perceptron**:



A **Perceptron** is a type of network structure consisting of **nodes** (circles in the diagram) connected to each other by **edges** (arrows in the diagram). The nodes appear in vertical

layers

Preview: Docs Loading link description

, connected from left to right.

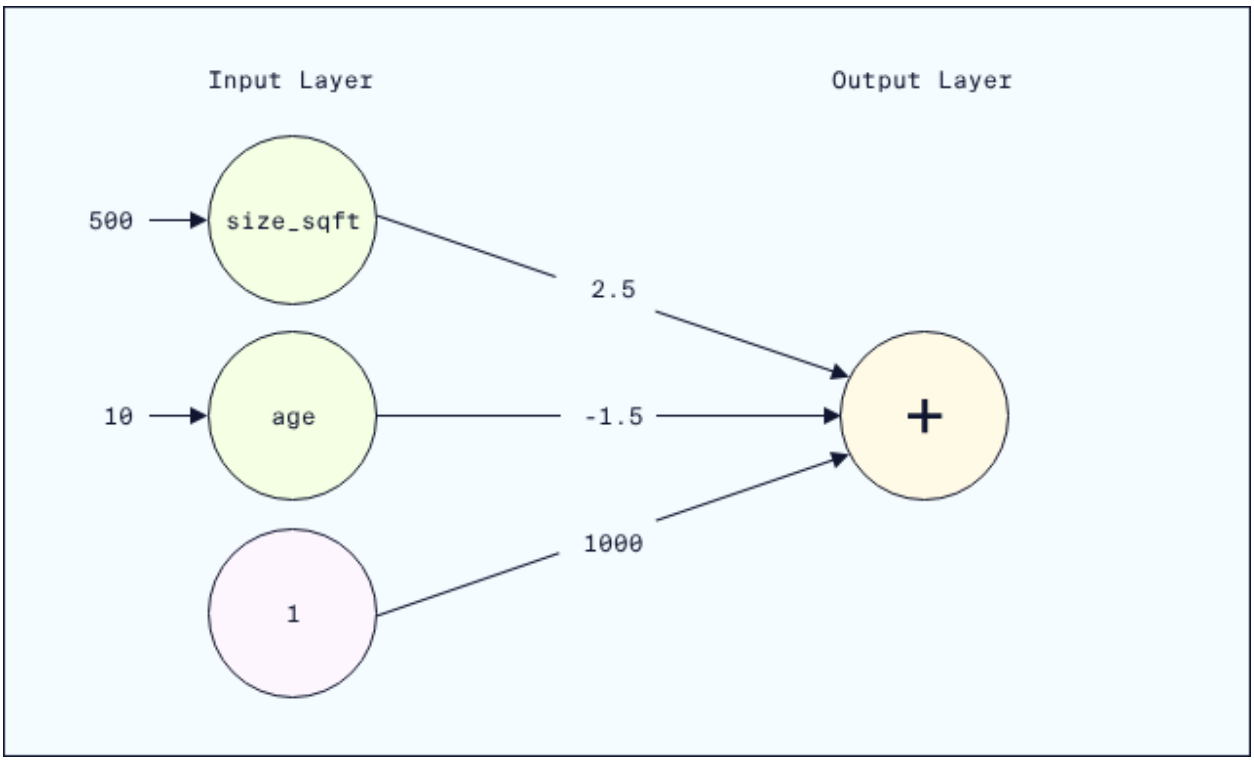
What makes a network a Perceptron is that it has one set of input nodes leading to a single output node.

Let's walk through how this Perceptron represents the original equation.

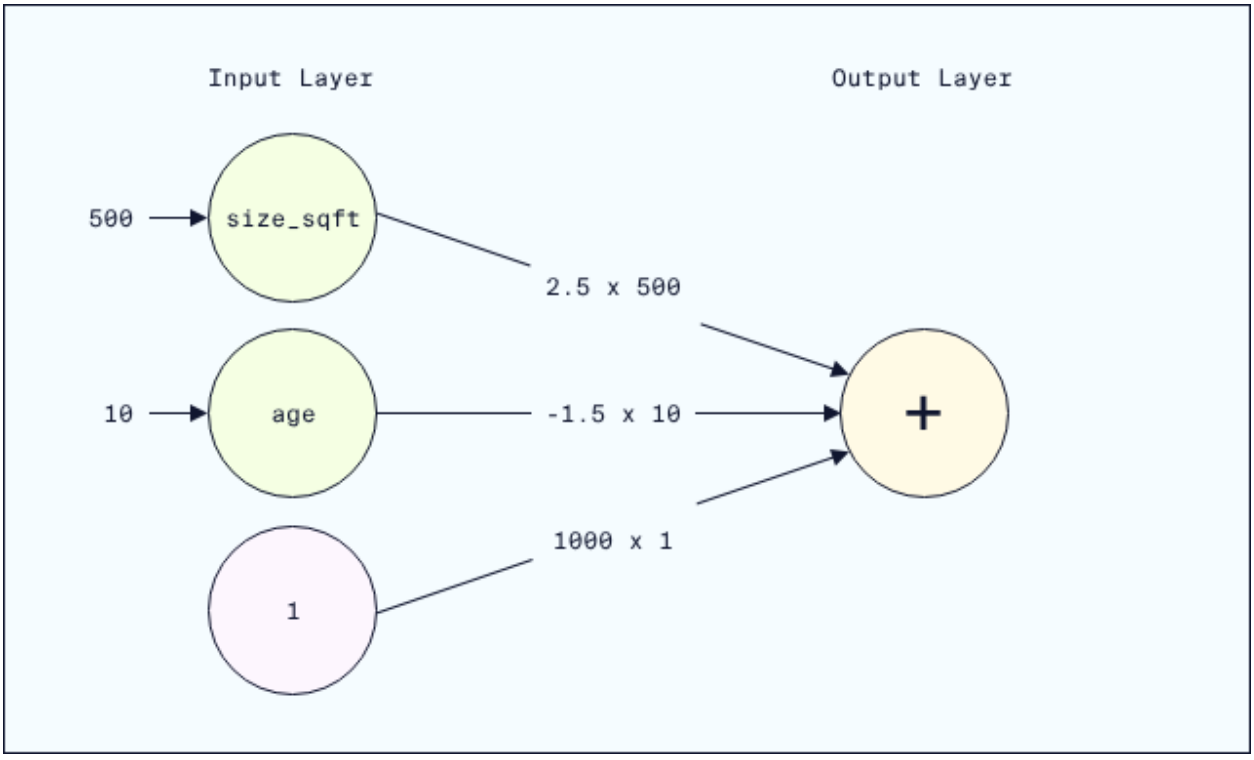
Suppose we want to predict the rent price for a `10` year old apartment with `500` square footage.

First, we send each input value (`500` for square feet and `10` for age) into the input nodes:

Then, each input value flows forward into the edges, and is multiplied by the corresponding weight:



Lastly, the weighted values flow into the output node and are added together:



This final equation

$$rent=2.5\times500-1.5\times10+1000\times1$$

$$rent=2.5\times500-1.5\times10+1000\times1$$

is exactly the same as what we got with the original linear equation!

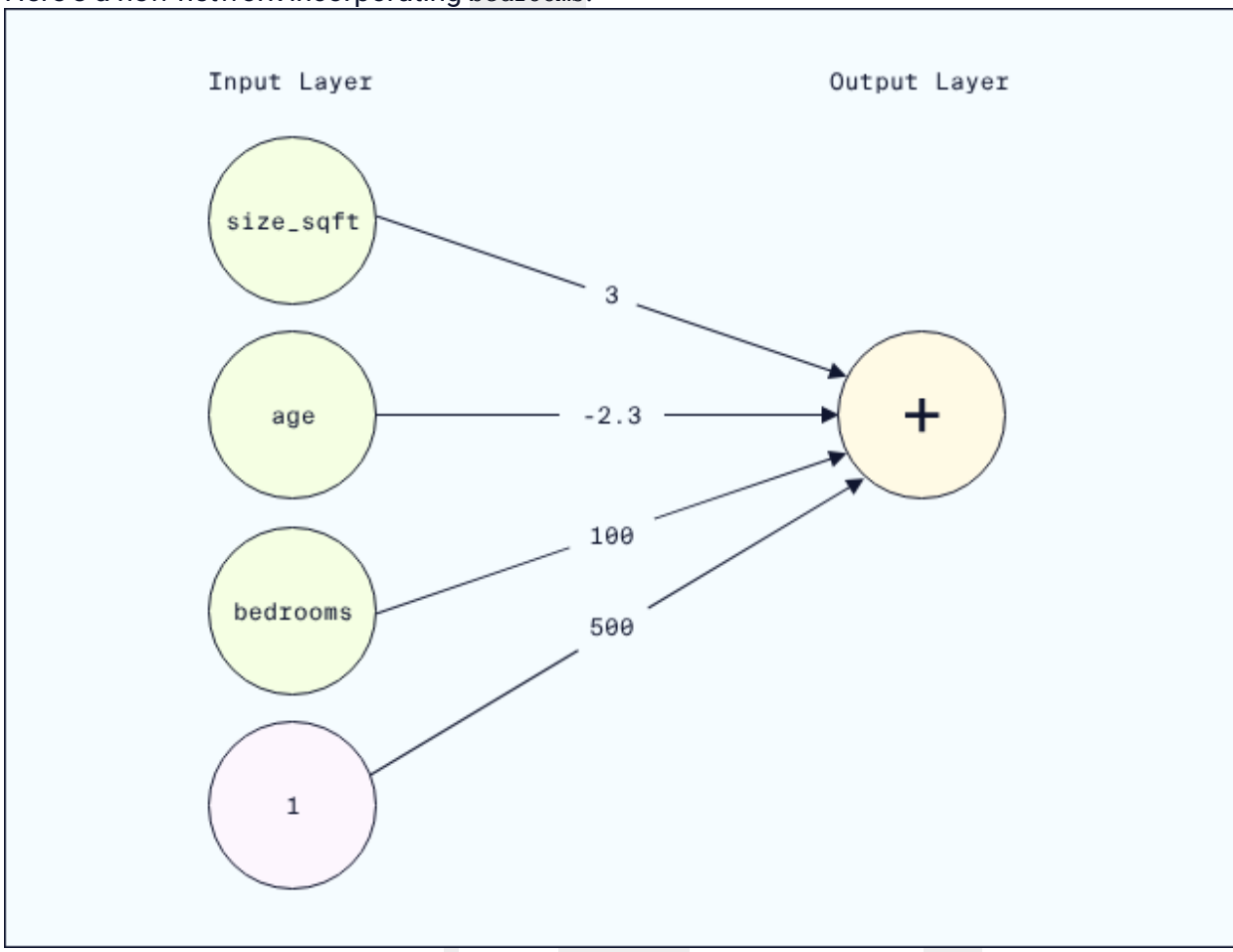
Note that eventually we'll stop thinking of the bias as coming from its own input node (`1`) with weight (`1000`). This is helpful to see how these networks can exactly duplicate linear equations, but once we start working with PyTorch the biases will simply be tacked on to the sum of the "real" inputs.

Instructions

Checkpoint 1 Enabled

1.

First, run through the Python code in the **Example** cell.
Now, let's add an additional input feature `bedrooms` to our linear regression perceptron which corresponds to the number of bedrooms in the apartment.
Here's a new network incorporating `bedrooms`:



This new network has a weight of `3` for the `size_sqft` input, a weight of `-2.3` for the `age` input, a weight of `100` for the `bedrooms` input, and a weight of `500` for the bias.
Using this updated network, predict the rent for an apartment with

`size_sqft` equal to `1250.0`

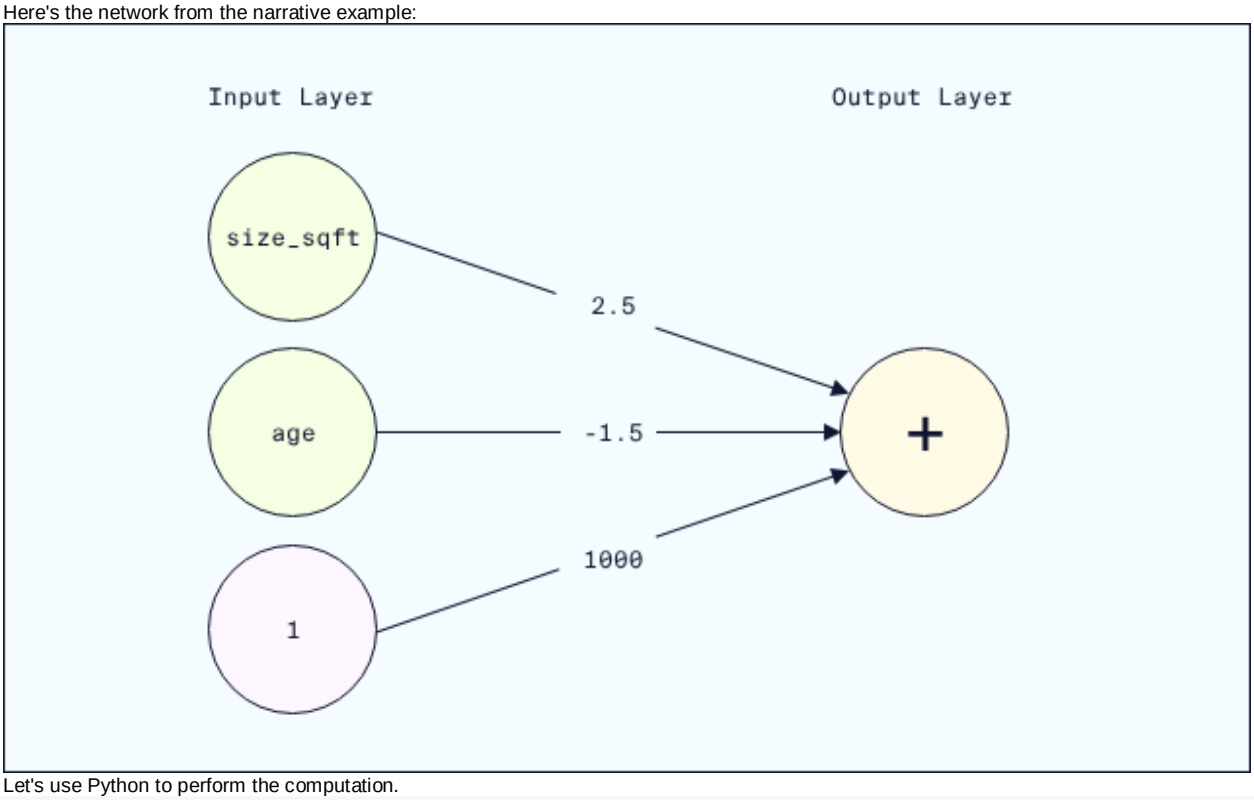
`age` equal to `15.0` years

number of `bedrooms` equal to `2.0`

Save the predicted rent price to the variable `weighted_sum`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.

Start with the code from the example. Add `bedrooms` to the input, and compute `weighted_bedrooms`. Then, add `weighted_bedrooms` to the weighted sum computation.
Don't forget to adjust the weights to match the new network diagram!

Here's the network from the narrative example:

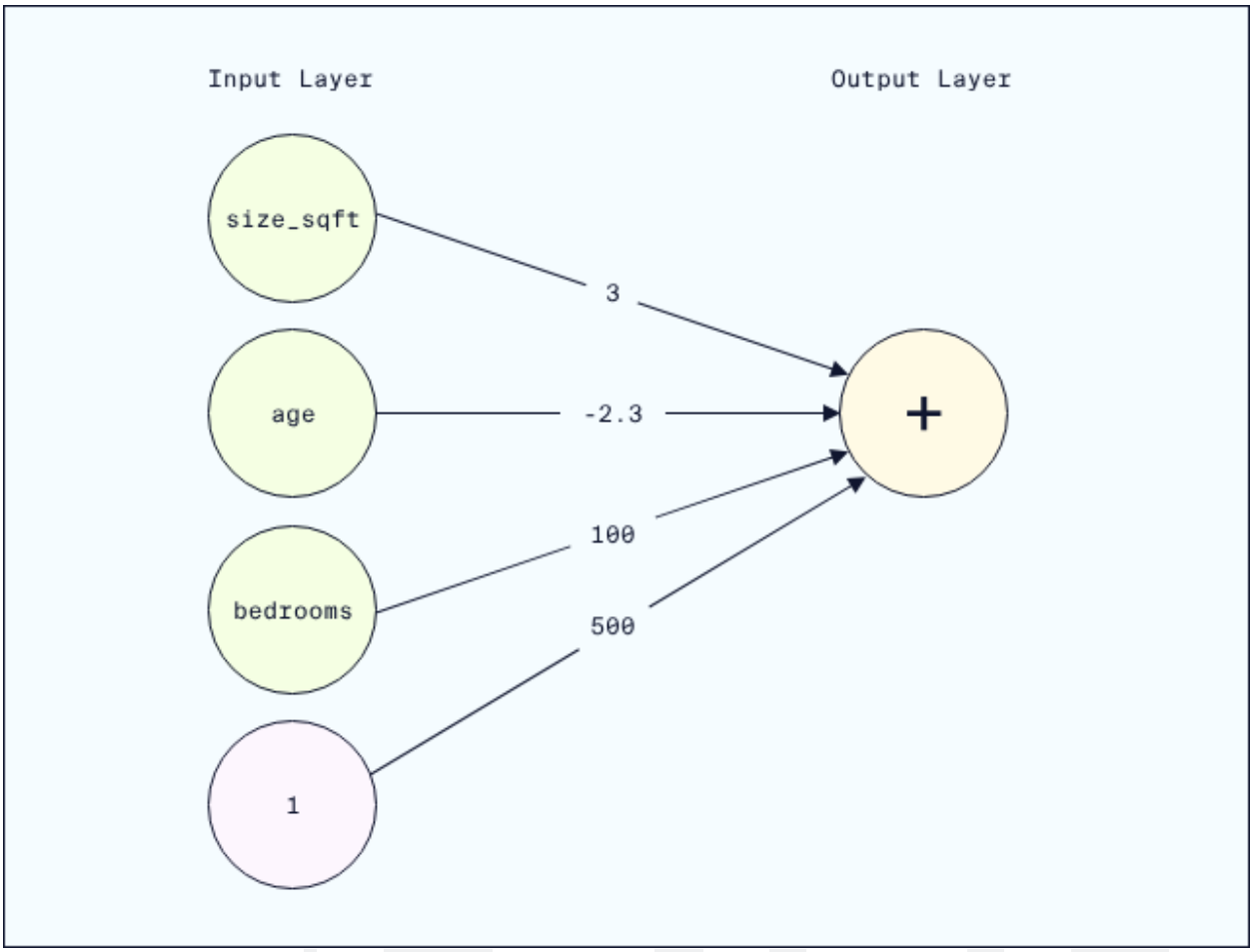


Let's use Python to perform the computation.

```
# Define the inputs
size_sqft = 500.0
age = 10.0
bias = 1

# The inputs flow through the edges, receiving weights
weighted_size = 2.5 * size_sqft
weighted_age = -1.5 * age
weighted_bias = 1000 * bias

# The output node adds the weighted inputs
weighted_sum = weighted_size + weighted_age + weighted_bias

# Generate prediction
print("Predicted Rent:", weighted_sum)
```

Predicted Rent: 2235.0

Checkpoint 1 / 1
Let's add an additional input feature `bedrooms` to our linear regression perceptron which corresponds to the number of bedrooms in the apartment.
Here's a new network incorporating `bedrooms`:



This new network has a weight of `3` for the `size_sqft` input, a weight of `-2.3` for the `age` input, a weight of `100` for the `bedrooms` input, and a weight of `500` for the bias.
Using this updated network, predict the rent for an apartment with

- `size_sqft` equal to `1250.0`
- `age` equal to `15.0` years
- number of `bedrooms` equal to `2.0`

Save the predicted rent price to the variable `weighted_sum`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##

# Define the inputs
size_sqft = 1250.0
age = 15.0
bias = 1
bedrooms = 2

# The inputs flow through the edges, receiving weights
weighted_size = 3 * size_sqft
weighted_age = -2.3 * age
weighted_bedrooms = 100 * bedrooms
weighted_bias = 500 * bias

# The output node adds the weighted inputs
weighted_sum = weighted_size + weighted_age + weighted_bedrooms + weighted_bias

# Generate prediction
print("Predicted Rent:", weighted_sum)
```

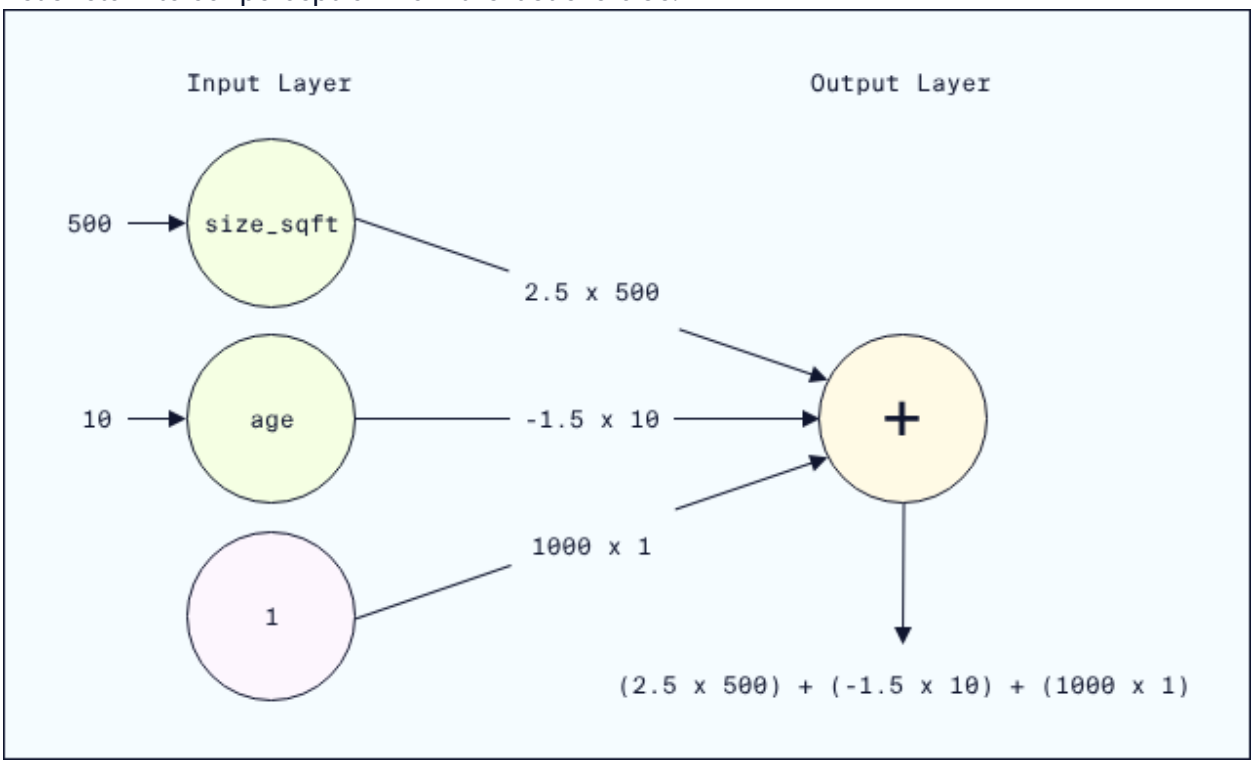
Predicted Rent: 4415.5

Activation Functions

In the last exercise, the output of the perceptron was a linear equation. If we stopped developing neural networks here, they would just be fancy ways of doing linear regression. But they are so much more!

One of the ways neural networks move beyond linear regression is by incorporating non-linear **activation functions**. These functions allow a neural network to model nonlinear relationships within a dataset, which are very common and cannot be modeled with linear regression.

Let's return to our perceptron from the last exercise:



The output node produces a rent prediction in a two step process:

- receive the weighted inputs
- add them up

The result of this process is a linear equation.

Activation functions take the linear equation as input, and modify it to introduce non-linearity.

The process *with* an activation function is

- receive the weighted inputs
- add them up (to produce the same linear equation as before)
- apply an activation function

ReLU Activation Function

One of the most common

activation functions

Preview: Docs Loading link description

used in neural networks is called **ReLU**.

If a number is negative, ReLU returns `0`. If a number is positive, ReLU returns the number with no changes.

For example,

```
ReLU(-1)
# output: 0, since -1 is a negative number

ReLU(.5)
# output: .5, since .5 is not negative
```

Copy to Clipboard

If the weighted inputs coming into a ReLU node are `3` and `-4`, the output would be calculated by

receiving the weighted inputs: `3` and `-4`
adding them together: `3-4 = -1`
applying ReLU: `ReLU(-1) = 0`
output: `0`

It might feel a bit strange that we're just turning negative numbers into 0. But this actually helps networks model datasets, by having different sets of nodes be "active" (or nonzero) for different inputs. This is one of the reasons neural networks were originally thought to model the human brain, where different sets of neurons fire under different circumstances (although, as we discussed, current evidence suggests neural networks don't really behave like the human brain.) Zeroing out negative numbers also helps when training neural networks using gradient descent (which we'll discuss later in the course).

Other activation functions

There are
many other activation functions
Preview: Docs Loading link description
used in different neural network applications. For example, the
sigmoid activation function
Preview: Docs Loading link description
only outputs values between 0 and 1, and is common in classification problems.

We'll often use icons to indicate the activation function in diagrams, based on the shape of the activation function when plotted:

- ReLU:
- Sigmoid:
- No activation function, just the weighted sum:

Don't worry too much about learning all the activation functions. As we cover different applications of neural networks, we'll learn about the standard activation functions for those networks.

Instructions

Checkpoint 1 Passed

1.

Calculate

a. ReLU(-3)
b. ReLU(0)
c. ReLU(3)

Assign your answers as integers to `answer_1`, `answer_2`, and `answer_3` in order.
Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.

ReLU turns negative numbers into 0, and returns other numbers without modification.
Checkpoint 2 Passed

2.

Suppose the weighted inputs to a node are

a. -3.5
b. 3

Compute the output of the node in two ways:

a. **Linear output / no activation**: compute the output of the node using no activation function. Assign your answer to the variable `Linear_output`.
b. **ReLU activation**: use the `ReLU` function we've defined for you to compute the output of the node. Assign your answer to the variable `ReLU_output`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.

Linear output is computed in two steps:

a. receive the weighted inputs
b. add them up

Any activation function, like ReLU, is applied to the result of Step 2.
Checkpoint 3 Passed

3.

Suppose the weighted inputs to a node are

a. -2
b. 1
c. .5

We've tried to calculate the ReLU output in the cell below, but we've made an error. Can you fix it? The correct output is `0`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.
The activation function is applied after adding together the weighted inputs.

Calculate

1. ReLU(-3)
2. ReLU(0)
3. ReLU(3)

Assign your answers as integers to `answer_1`, `answer_2`, and `answer_3` in order.
Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
answer_1 = 0
answer_2 = 0
answer_3 = 3

# show output
print('ReLU(-3) = ' + str(answer_1))
print('ReLU(0) = ' + str(answer_2))
print('ReLU(3) = ' + str(answer_3))
```

```
ReLU(-3) = 0
ReLU(0) = 0
ReLU(3) = 3
```

Checkpoint 2/3
Suppose the weighted inputs to a node are

1. -3.5
2. 3

Compute the output of the node in two ways:

1. **Linear output / no activation**: compute the output of the node using no activation function. Assign your answer to the variable `Linear_output`.
2. **ReLU activation**: use the `ReLU` function we've defined for you to compute the output of the node. Assign your answer to the variable `ReLU_output`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.

```
# define the ReLU function
def ReLU(x):
    return max(0,x)

## YOUR SOLUTION HERE ##
Linear_output = -3.5 + 3
ReLU_output = ReLU(Linear_output)

# show output
print('ReLU node output: ' + str(ReLU_output))
print('Linear node output: ' + str(Linear_output))
```

ReLU node output: 0
Linear node output: -0.5

Checkpoint 3/3

Suppose the weighted inputs to a node are

1. -2
2. 1
3. .5

We've tried to calculate the ReLU output in the cell below, but we've made an error. Can you fix it? The correct output is `0`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! See the `Jupyter Help` toggle at the top of the notebook for more details.

```
# define the ReLU function
def ReLU(x):
    return max(0,x)

## FIX OUR SOLUTION ##
ReLU_output = ReLU(-2 + 1 + .5)

# show output
ReLU_output
```

0

Multi-Layer Networks

So far, our networks have all had:

one input layer
one output layer

Activation functions help introduce nonlinearity, but to really model arbitrarily complex datasets, we need a more complex multi-layer network structure. We also need to train the network on our data, so it can learn the complex relationships within the dataset.

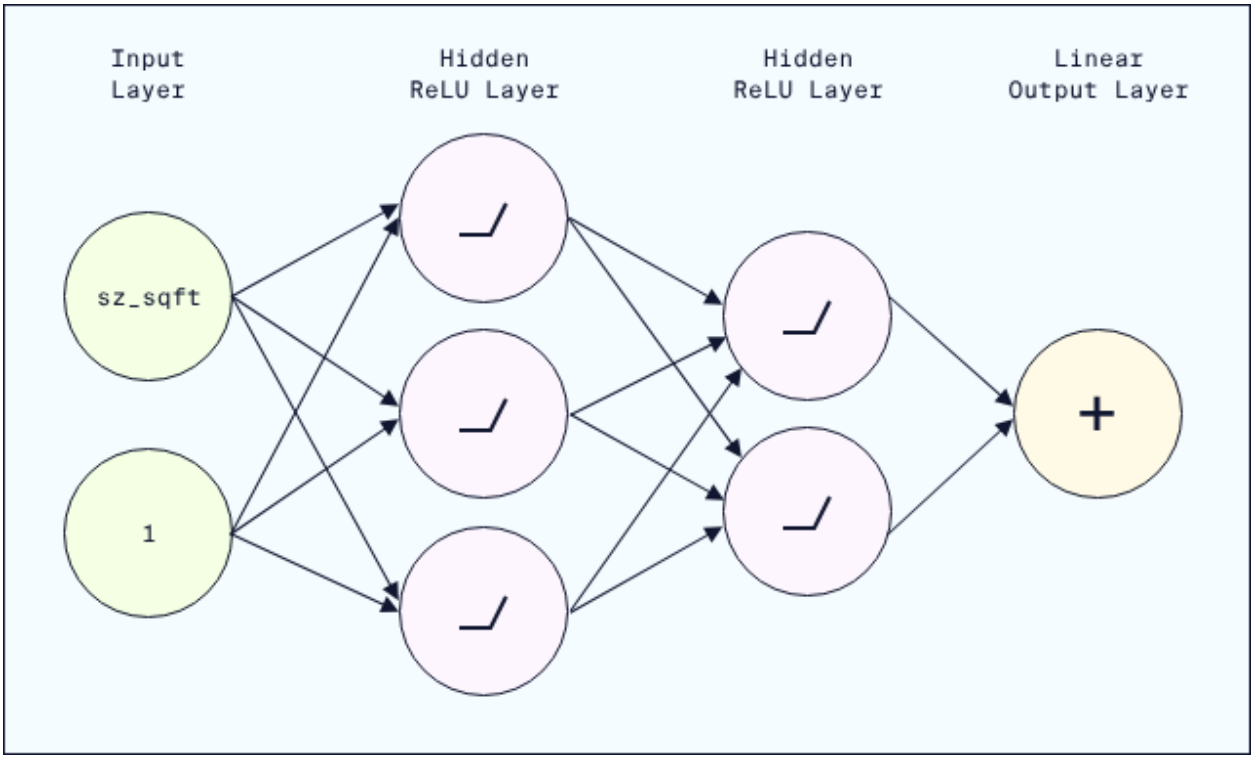
Hidden Layers

Hidden

layers

Preview: Docs Loading link description

are layers of nodes introduced between the input and output layer. Here's an example of a neural network with two hidden layers:



Input data flows through the network from the input layer, to the hidden layers, and on to the output. Hidden layer nodes behave somewhat like output nodes have so far: receiving inputs and computing outputs. In other words, each node of the hidden layer functions like a Perceptron.

To be precise, each node in a hidden layer

- receives weighted inputs from the nodes in the prior layer
- adds up all those weighted inputs with a bias term
- (optionally) applies an activation function to the weighted sum
- sends the result to every node in the next layer

Generally speaking, every node in a particular hidden layer will apply the same activation function.

From now on, we'll stop creating a separate bias node. While helpful as an initial model, it will be easier to just remember that every weighted sum also includes a bias term.

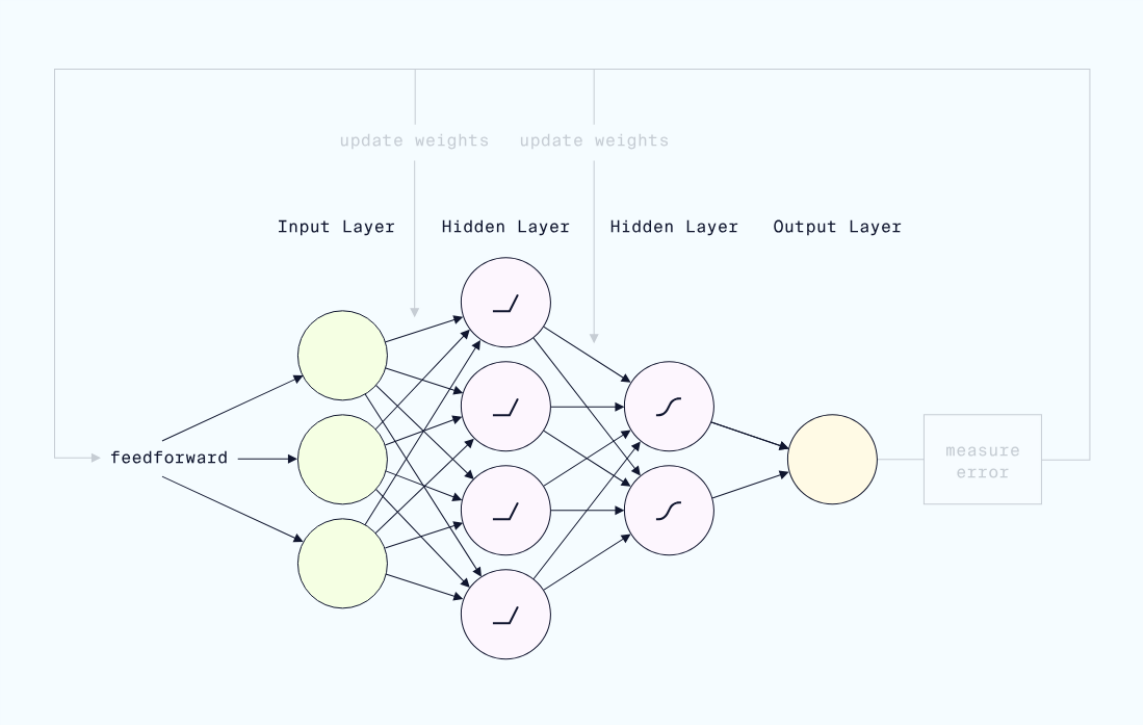
The Training Process

The training process has roughly four steps:

- **Forward pass / feedforward**: we feed input data through the network from layer to layer and calculate the final output value
- **Loss**: we measure how close (or far!) the network's predictions are to the actual values
- **Backward pass / backpropagation**: we apply an optimization algorithm to go back and update the weights and biases of the network, to try to improve the network's performance
- **Iterate**: we repeat this process over and over, checking each time if the loss (or error) is going down

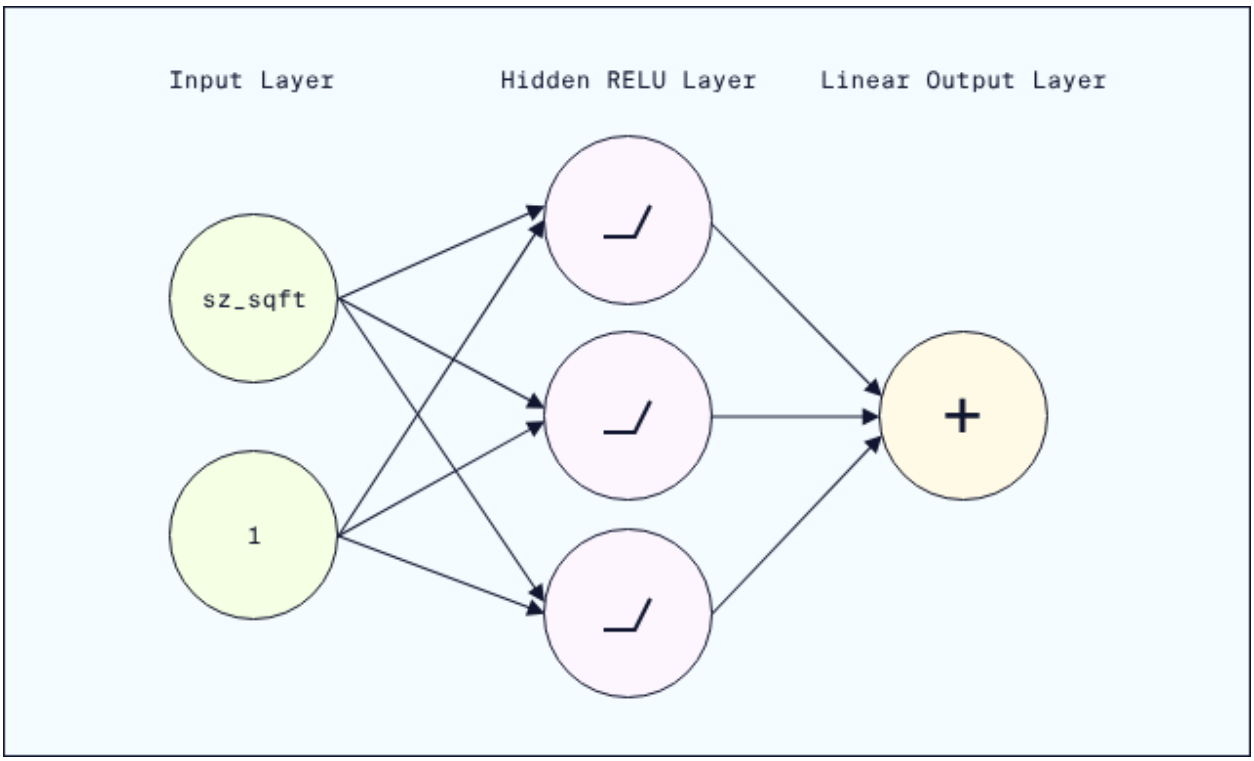
Instructions

We've included a detailed walkthrough of the feedforward and training process in the learning environment. When you are ready to start implementing this in PyTorch, select `Next`!



Build a Sequential Neural Network

Now that we know the basic structure of a neural network, let's build one in PyTorch using PyTorch's `Sequential` container.
Suppose we want to create the following neural network:



This network has

- input layer: two nodes
- one hidden layer: three nodes with ReLU activation
- output layer: one node

We can build this network using two PyTorch methods inside `nn.Sequential`:

- `nn.Linear()` for defining the layers and performing the linear calculations (weighted inputs plus bias)
- `nn.ReLU()` for applying the activation function

Here's how we'd build the example network:

```
model = nn.Sequential(
    nn.Linear(2,3),
    nn.ReLU(),
    nn.Linear(3,1)
)
```

Copy to Clipboard

Let's go through this step by step:

- `nn.Linear(2,3)` connects the two input nodes to the three hidden nodes using our standard weights-and-bias linear calculation
- `nn.ReLU()` applies the ReLU activation function to that linear computation
- `nn.Linear(3,1)` connects the ReLU output from the three hidden nodes to the one output node (again, using our usual linear calculation)

Note that the connections between layers must be properly aligned. Once we have defined `nn.Linear(2,3)` as the first layer, the next `nn.Linear()` must start with 3 nodes.

Initially, each instance of `nn.Linear()` will use randomly generated weights and bias. In the training process, which we'll cover later, the neural network will update these to improve its predictions.

Running Feedforward

The first step of neural network training and predicting is the feedforward process. To run through the feedforward process with our `Sequential` model, we need to pass a tensor as input to `model`.

Typically our input tensors will be two-dimensional, consisting of:

- **rows**: individual examples (in our case, each row is an apartment)
- **columns**: individual features (in our case, each column is an apartment property, like size)

Note that statisticians call examples **observations**.

Suppose the two input nodes in our model correspond to building age and number of bedrooms.

Let's create an input tensor with a couple apartments and run the feedforward process. Note that typically input tensors are stored in a variable named `X` (this is a convention from mathematics that main ML engineers follow). The input tensor is usually capitalized (`X` and not `x`) to indicate that it is a two-dimensional matrix of rows and columns.

```
# create apartment data
apts = np.array(
    [[100,3], # 100 years old, 3 bedrooms
    [50,4]]) # 50 years old, 4 bedrooms

# convert to a tensor
X = torch.tensor(apts, dtype=torch.float)

# run feedforward
model(X)

# output: the result of a feedforward through the network layers
>>>tensor([[-23.0715],
        [-11.8710]], grad_fn=<AddmmBackward0>)
```

Copy to Clipboard

We can interpret this output by associating each row of output to the same row of the input tensor:

- input row `[100,3]` corresponds to output/prediction of `-23.0715`.
- input row `[50,4]` corresponds to output/prediction of `-11.8710`.

Of course, as predictions these make no sense (though we'd love to have negative rent!) But we haven't trained this model at all, so that isn't surprising.

We'll discuss the last piece of this output (`grad_fn=<AddmmBackward0>`) when we start training models with backpropagation.

Instructions

Checkpoint 1 Passed

1.

Use `nn.Sequential` to create a neural network model with

input layer: three nodes

hidden layer: eight nodes, with ReLU activation
output layer: one node

Assign your network to the variable `model`.

Because `nn.Sequential` will randomly initialize weights and biases, we've set a **random seed** in every Checkpoint code cell. This is generally a good practice that means the initial weights and biases are the same every time we run the same code. Do not edit the seed value `42` as that is needed for our testing code.

Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

The skeleton code is

```
model = nn.Sequential(
    # define layers and activation functions
)
```

Copy to Clipboard

To connect a layer with 5 nodes to a layer with 23 nodes, for example, we'd use

```
nn.Linear(5,23)
```

Copy to Clipboard

The ReLU activation function is `nn.ReLU()`.

Checkpoint 2 Passed

2.

Re-create the model from the prior checkpoint. This time, add a second hidden layer with

four nodes
`nn.Sigmoid()` as the activation function

Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

You will need to

copy the `nn.Sequential` model from Checkpoint 1 as the model for Checkpoint 2
modify the last `nn.Linear()` call to connect to a new 4-node hidden layer
add the sigmoid activation function
connect the new 4-node hidden layer to the one output node

Checkpoint 3 Passed

3.

First, follow the instructions to run the **Dataset Import** cell to import our dataset and convert it to a tensor.

Great! Now that we have our data stored in `X`, let's create a neural network.

In the Checkpoint 3 code cell, we've already created a neural network model. It follows a fairly common neural network structure, where each hidden layer is halved in size as we feed forward to the output.

Run the feedforward process of `model` on `X`. Assign the result to the variable `predicted_rent`.

We've added code to preview the first five predicted rents.

Of course, these initial predictions will not be accurate at all. Without training, the

feedforward is just applying randomly generated weights and biases! We'll train these on the actual rent values in a later exercise.
To feedforward, pass the tensor as an input to `model()`.

Having trouble testing your work? Double-check that you have followed the steps below to write, run, save, and test your code!
Click here for a walkthrough GIF of the steps below
Run all initial cells to import libraries and datasets. Then follow these steps for each question:

1. Add your solution to the cell with `## YOUR SOLUTION HERE ##`.
2. Run the cell by selecting the `Run` button or the `Shift`+`Enter` keys.
3. Save your work by selecting the `Save` button, the `command`+`s` keys (Mac), or `control`+`s` keys (Windows).
4. Select the `Test Work` button at the bottom left to test your work.

Setup
Run the cell below to import NumPy, pandas, and PyTorch.

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
```

Checkpoint 1/3
Use `nn.Sequential` to create a neural network model with

- **input layer**: three nodes
- **hidden layer**: eight nodes, with ReLU activation
- **output layer**: one node

Assign your network to the variable `model`.
Because `nn.Sequential` will randomly initialize weights and biases, we've set a **random seed** in every Checkpoint code cell. This is generally a good practice that means the initial weights and biases are the same every time we run the same code. Do not edit the seed value `42` as that is needed for our testing code.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

## YOUR SOLUTION HERE ##
model = nn.Sequential(
    nn.Linear(3,8),
    nn.ReLU(),
    nn.Linear(8,1)
)

# show model details
model
```

```
Sequential(
  (0): Linear(in_features=3, out_features=8, bias=True)
  (1): ReLU()
  (2): Linear(in_features=8, out_features=1, bias=True)
)
```

Breakdown of this output
The output lists the sequential structure of the network. For example, the line
`(0): Linear(in_features=3, out_features=8, bias=True)`
indicates that we begin with a linear calculation from three nodes (the `in_features`) to eight nodes (the `out_features`). The line `bias=True` indicates that we are including a bias term (even though we aren't explicitly creating a bias node).

Checkpoint 2/3
Re-create the model from the prior checkpoint. This time, add a second hidden layer with

- four nodes
- `nn.Sigmoid` as the activation function

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

## YOUR SOLUTION HERE ##
model = nn.Sequential(
    nn.Linear(3,8),
    nn.ReLU(),
    nn.Linear(8,4),
    nn.Sigmoid(),
    nn.Linear(4,1)
)

# show model details
model
```

```
Sequential(
  (0): Linear(in_features=3, out_features=8, bias=True)
  (1): ReLU()
  (2): Linear(in_features=8, out_features=4, bias=True)
  (3): Sigmoid()
  (4): Linear(in_features=4, out_features=1, bias=True)
)
```

Dataset Import

Let's create a model and feedforward all of our real Streeteasy data.

First, run the next code cell to import the Streeteasy data and convert the `size_sqft`, `bedrooms`, and `building_age_yrs` columns into a PyTorch tensor.

We will use these three columns to try to predict rent!

```
# load pandas DataFrame
apartments_df = pd.read_csv("streeteasy.csv")

# create a numpy array of the numeric columns
apartments_numpy = apartments_df[['size_sqft', 'bedrooms', 'building_age_yrs']].values

# convert to an input tensor
X = torch.tensor(apartments_numpy,dtype=torch.float32)

# preview the first five apartments
X[:5]
```

```
tensor([[4.8000e+02, 0.0000e+00, 1.7000e+01],
        [2.0000e+03, 2.0000e+00, 9.6000e+01],
        [1.0000e+03, 3.0000e+00, 1.0600e+02],
        [9.1600e+02, 1.0000e+00, 2.9000e+01],
        [9.7500e+02, 1.0000e+00, 3.1000e+01]])
```

Checkpoint 3/3

Now that we have our data stored in `X`, let's create a neural network.

In the Checkpoint 3 code cell, we've already created a neural network model. It follows a fairly common neural network architecture, where each hidden layer is halved in size as we feed forward to the output.

Run the feedforward process of `model` on `X`. Assign the result to the variable `predicted_rents`.

We've added code to preview the first five predicted rents. What is the predicted rent for the first apartment in the dataset?

Our answer

Of course, these initial predictions will not be accurate at all. Without training, the feedforward is just applying randomly generated weights and biases! We'll train these on the actual rent values in a later exercise.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

# define the neural network
model = nn.Sequential(
    nn.Linear(3,16),
    nn.ReLU(),
    nn.Linear(16,8),
```

```
    nn.ReLU(),
    nn.Linear(8,4),
    nn.ReLU(),
    nn.Linear(4,1)
)

## YOUR SOLUTION HERE ##
predicted_rent = model(X)

# show output
predicted_rent[:5]
```

```
tensor([[ -6.9229],
        [-29.8163],
        [-16.0748],
        [-13.2427],
        [-14.1096]], grad_fn=<SliceBackward0>)
```

Build a Neural Network Class

While `nn.Sequential` is pretty useful for creating neural networks, often AI developers need to create non-sequential types of neural networks using **object-oriented programming (OOP)**. Using OOP gives developers much more control over neural networks. With `Sequential`, we can only feed data from one layer to the next. Building in logic to skip certain
layers
Preview: Docs Loading link description
or to loop others can improve neural network performance. For that, we need the flexibility of OOP.

In this exercise, we're assuming no familiarity with OOP. Don't worry if the syntax isn't completely familiar or clear, for now. Our goal for now is to help you navigate OOP neural network code and prepare for more advanced courses.

Let's start by building a familiar sequential network with OOP.

1. Create the `NN_Regression` Class

In the prior exercise, we created sequential neural networks using the syntax

```
model = nn.Sequential(<inputs>)
```

Copy to Clipboard

In this syntax, `nn.Sequential` refers to a type of neural network (sequential neural network). Types of things in OOP are called **classes**. The variable `model` is a specific `Sequential` neural network, called an **instance** of the **class**.

We can create our own classes (or types) of PyTorch neural networks using the syntax

```
class NN_Regression(nn.Module):
```

Copy to Clipboard

2. Initialize the Network Components

Inside the `class` declaration, we need to **initialize** all the layers and
activation functions
Preview: Docs Activation functions are mathematical functions that introduce non-linearity into the model, enabling neural networks to learn complex patterns from data.

we plan to use. We can think of this step as “gathering all the ingredients” for a recipe.

```
def __init__(self):
    super(NN_Regression, self).__init__()
    # initialize the layers
    self.layer1 = nn.Linear(3, 16)
    self.layer2 = nn.Linear(16, 8)
    self.layer3 = nn.Linear(8, 4)
    self.layer4 = nn.Linear(4, 1)

    # initialize activation functions
    self.relu = nn.ReLU()
```

Copy to Clipboard

Note that the `self` syntax just allows us to reference these variables, like `self.layer1`, in the next section of defining the class.

3. Define the Forward Pass

Now that we’ve gathered the ingredients, we need to describe how to combine them in order to perform the feedforward operation.

We’ll create a `forward` method that dictates the flow of how an input data tensor `x` is passed from layer to layer through the network. Within this method, we can use all the ingredients we already gathered, like `self.layer1` and `self.relu`.

```
def forward(self, x):
    # define the forward pass
    x = self.layer1(x)
    x = self.relu(x)
    x = self.layer2(x)
    x = self.relu(x)
    x = self.layer3(x)
    x = self.relu(x)
    x = self.layer4(x)
    return x
```

Copy to Clipboard

Here, the syntax `x = self.layer1(x)`, for example, takes the input tensor `x` and passes it through the input layer to the next hidden layer.

And that’s it! We now have a new type of PyTorch neural network called `NN_Regression`.

4. Instantiate the Model

What we’ve created so far is just a type of neural network. Just like we had to call `model = nn.Sequential()` to create an instance of the `Sequential` class, we’ll need to run

```
model = NN_Regression()
```

Copy to Clipboard

to create an instance of an `NN_Regression` neural network!

Instructions

Checkpoint 1 Passed

1.

In the Checkpoint 1 code cell we’ve included code to

create the `NN_Regression` class from the narrative
create an input tensor of apartment data
feedforward to make predictions

Run the cell. The output from this network should be the same as the Sequential network from the last exercise! Once again, this network is completely untrained, so it isn't surprising that the "predictions" are very bad.

Note: even though you haven't written any code in this cell, still save and select `Test Work` to move on!

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Checkpoint 2 Passed

2.

We've created a new neural network class called `OneHidden` with:

two node input layer
four node hidden layer
one node output layer

This class does not yet have any instructions for feedforward. Try running the cell without making any changes to the code. Without any linear layers or activation functions, the input tensor should be output as is.

In the `forward` method, define a feedforward that takes the input `x` and feeds it through:

`self.layer1`
`self.relu`
`self.layer2`

Run the cell again to see the new predictions!

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Checkpoint 3 Passed

3.

Let's look at one way defining classes can be helpful.

We've included the same `OneHidden` class from Checkpoint 2 in the Checkpoint 3 code cell below. But we've updated the line

```
def __init(self):
```

Copy to Clipboard

to have another input:

```
def __init__(self, numHiddenNodes):
```

Copy to Clipboard

This input, `numHiddenNodes`, determines the number of hidden nodes in the hidden layer. For example, within the `__init__` method, the line

```
self.layer1 = nn.Linear(2, numHiddenNodes)
```

Copy to Clipboard

uses `numHiddenNodes` to define the connections between the input and hidden layer.

Below `## YOUR SOLUTION HERE ##`, pass the input `4` to `OneHidden()` in the line `model = OneHidden()`.
Run the cell. The output should be the same as for Checkpoint 2, where we hand coded the value `4` for the hidden layer.
Now, change the number of nodes in the hidden layer to `10` and run the cell again. How does the output change?
Save the notebook after running with `10` hidden layer nodes and test your work.
Don't forget to run the cell and save the notebook before selecting `Test Work`!
Open the `Jupyter Help` toggle at the top of the notebook for more details.

Update the line

```
model = OneHidden()
```

Copy to Clipboard

to

```
model = OneHidden(10)
```

Copy to Clipboard

Don't forget to run and save the notebook after this last update, and then select `Test Work`.

Setup

Run the following cell to import NumPy, pandas, and PyTorch.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
```

Checkpoint 1/3: Create a Neural Network in PyTorch Using OOP

In the Checkpoint 1 code cell we've included code to

1. create the `NN_Regression` class from the narrative
2. create an input tensor of apartment data
3. feedforward to make predictions

Run the cell. The output from this network should be the same as the Sequential network from the last exercise! Once again, this network is completely untrained, so it isn't surprising that the "predictions" are very bad.

Note: even though you haven't written any code in this cell, still save and select `Test Work` to move on!

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

## create the NN_Regression class
class NN_Regression(nn.Module):
    def __init__(self):
        super(NN_Regression, self).__init__()
        # initialize layers
        self.layer1 = nn.Linear(3, 16)
        self.layer2 = nn.Linear(16, 8)
        self.layer3 = nn.Linear(8, 4)
        self.layer4 = nn.Linear(4, 1)

        # initialize activation functions
        self.relu = nn.ReLU()

    def forward(self, x):
        # define the forward pass
```

```
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        x = self.relu(x)
        x = self.layer3(x)
        x = self.relu(x)
        x = self.layer4(x)
        return x

## create an instance of NN_Regression
model = NN_Regression()

## create an input tensor

apartments_df = pd.read_csv("streeteasy.csv")
numerical_features = ['size_sqft', 'bedrooms', 'building_age_yrs']
apartments_tensor = torch.tensor(apartments_df[numerical_features].values, dtype=torch.float)

## feedforward to predict rent
predicted_rents = model(apartments_tensor)

## show output
predicted rents[:5]
```

tensor([[-6.9229],
[-29.8163],
[-16.0748],
[-13.2427],
[-14.1096]], grad_fn=<SliceBackward0>)

Checkpoint 2/3

We've created a new neural network class called `OneHidden` with:

- a two node input layer
- a four node hidden layer
- a one node output layer

This class does not yet have any instructions for feedforward. Try running the cell without making any changes to the code. Without any linear layers or activation functions, the input tensor should be output as is.

In the `forward` method, define a feedforward that takes the input `x` and feeds it through:

1. `self.layer1`
2. `self.relu`
3. `self.layer2`

Run the cell again to see the new predictions!

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

## create the NN Regression class
class OneHidden(nn.Module):
    def __init__(self):
        super(OneHidden, self).__init__()
        # initialize layers
        self.layer1 = nn.Linear(2, 4)
        self.layer2 = nn.Linear(4, 1)

        # initialize activation functions
        self.relu = nn.ReLU()

    def forward(self, x):
        ## YOUR SOLUTION HERE ##
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x

## do not modify below this comment
```

```
# create an instance
model = OneHidden()

# create an input tensor
input_tensor = torch.tensor([3,4.5], dtype=torch.float32)

# run feedforward
predictions = model(input_tensor)

# show output
predictions
```

tensor([2.4422], grad_fn=<AddBackward0>)

Checkpoint 3 / 3: Make a prediction

Let's look at one way defining classes can be helpful.

We've included the same `OneHidden` class from Checkpoint 2 in the Checkpoint 3 code cell below. But we've updated the line

```
    def __init(self):
```

to have another input:

```
    def __init__(self,numHiddenNodes):
```

This input, `numHiddenNodes`, determines the number of hidden nodes in the hidden layer. For example, within the `__init__` method, the line

```
    self.layer1 = nn.Linear(2, numHiddenNodes)
```

uses `numHiddenNodes` to define the connections between the input and hidden layer.

Below `## YOUR SOLUTION HERE ##`, add the input `4` to the line `model = OneHidden` and run the cell. The output should be the same as for Checkpoint 2, where we hand coded the value `4` for the hidden layer.

Now, change the number of nodes in the hidden layer to `10` and run the cell again. How does the output change?

Save the notebook after running with `10` hidden layer nodes and test your work.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

## create the NN Regression class
class OneHidden(nn.Module):
   # add a new numHiddenNodes input
   def __init__(self, numHiddenNodes):
        super(OneHidden, self).__init__()
        # initialize layers
        # 3 input features, variable output features
        self.layer1 = nn.Linear(2, numHiddenNodes)
        # variable input features, 8 output features
        self.layer2 = nn.Linear(numHiddenNodes, 1)

        # initialize activation functions
        self.relu = nn.ReLU()

   def forward(self, x):
        ## YOUR SOLUTION HERE ##
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x

## YOUR SOLUTION HERE ##
model = OneHidden(10)

## do not modify below this comment

# create an input tensor
input_tensor = torch.tensor([3,4.5], dtype=torch.float32)

# run feedforward
predictions = model(input_tensor)

# show output
```

```
predictions
```
tensor([1.2633], grad_fn=<AddBackward0>)
Pretty cool, right? This is just scratching the surface of the power of defining our own neural network classes. But in the next exercise, we'll return to the problem of actually improving our predictions by training sequential neural networks.

The Loss Function

In the last two exercises, we predicted the rent of some apartments by feeding them through neural networks. The networks were untrained, so the predictions weren't very good.

Let's start improving those predictions!

To make better predictions, we first need to know how bad our current predictions are and how to track improvement as we train our model.

This is the role of the **loss function**. The loss function is a mathematical formula used to measure the error (also known as loss values) between the model predictions and the actual target values (sometimes called labels) the model is trying to predict.

Difference

Suppose we run a neural network on an apartment with $1000/mo rent, but the network predicts $500/mo. The simplest way to measure the loss in this case is to calculate the **difference**

$$500-1000=-500$$

$$500-1000=-500$$

The difference indicates that the prediction was 500 dollars below the actual rent.

For just one data point, the difference seems reasonable. But imagine we have a second apartment with rent $1500, but the model **overestimates** the rent as $2000. The difference in this case is

$$2000-1500=500$$

$$2000-1500=500$$

With one loss of `500` and another of `-500`, the average loss for the model is actually `0`! But that doesn't make sense, since this network isn't perfectly accurate.

The problem is that the negative loss cancelled out the positive loss. To fix this, we need to force the difference to always be positive.

Mean Squared Error

One of the most common

loss functions

Preview: Docs Loading link description

is Mean Squared Error (MSE). MSE makes differences positive by squaring them. To calculate MSE on our two example apartments, we would

calculate the diffferences: `500`, `-500`

square both: `500^2`, `(-500)^2`

take the average:

$$\frac{(500-1000)^2+(1500-1000)^2}{2}=250{,}000$$

$$\frac{(500-1000)^2}{2}+\frac{(1500-1000)^2}{2}$$

$$=250{,}000$$

A loss of `250,000` seems very high, but remember that we've squared all the individual differences. To help interpret MSE, we'll sometimes take the square root of the MSE:

$$\sqrt{\frac{250000}{2}}=500$$

An average loss of `500` makes a lot of sense in this case!

MSE in PyTorch

PyTorch has already implemented most of the common loss functions. To use PyTorch's implementation of MSE, we'd run

```
loss = nn.MSELoss()
```

```
Copy to Clipboard
```

Now that we've instantiated `loss`, we can compute the mean squared error by passing two inputs:

- the predicted values
- the actual target values

Just as we use `X` to stand for input features in a neural network, it is common in machine learning to use the variable `y` to stand for the target values. In this case, our target isn't two-dimensional, so we use lowercase `y`.

Let's calculate MSE for our two example apartments:

```
predictions = torch.tensor([500,2000],dtype=torch.float)
y = torch.tensor([1000,1500],dtype=torch.float)
print(loss(predictions,y))
```

```
Copy to Clipboard
```

Choosing a Loss Function

The loss function plays a key role in training, so it is important to select the right one. Sometimes it is worth experimenting with a few different loss functions, to see how each behaves.

For example, the squaring process in MSE emphasizes the largest differences between predicted and target values. Sometimes, this is helpful, but in other cases it can lead to overfitting. In those cases, instead of *squaring* differences we might choose to take the *absolute value* to produce positive values (this is called the **Mean Absolute Error**).

Instructions

Checkpoint 1 Passed

1.

Suppose we fed two apartments through a neural network with the result

predictions: 750, 1000

targets (actual rent): 1000, 900

We've tried to calculate MSE below, but we've made a mistake. Can you fix our calculation?

Don’t forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Checkpoint 2 Passed

2.

In Exercise 7 we built the neural network

```
model = nn.Sequential(
    nn.Linear(3,16),
    nn.ReLU(),
    nn.Linear(16,8),
    nn.ReLU(),
    nn.Linear(8,4),
    nn.ReLU(),
    nn.Linear(4,1)
)
```

Copy to Clipboard

When we fed our apartment data forward through the network, the first five predictions were

```
predictions = torch.tensor([-6.9229, -29.8163, -16.0748, -13.2427, -14.1096], dtype=torch.float)
```

Copy to Clipboard

The first five targets (actual rent values) were

```
y = torch.tensor([2550, 11500, 3000, 4500, 4795], dtype=torch.float)
```

Copy to Clipboard

Use PyTorch’s `nn.MSELoss()` function to compute the MSE of these five apartments. Assign the result to the variable `MSE`.

Don’t forget to run the cell and save the notebook before selecting `Test Work`!

Open the `Jupyter Help` toggle at the top of the notebook for more details.

First, assign `nn.MSELoss()` to a variable like

```
loss = nn.MSELoss()
```

Copy to Clipboard

The first input to `loss` is the predictions and the second input is the targets.

Checkpoint 3 Passed

3.

Let's make the result from Checkpoint 2 a bit more interpretable.
Compute the square root of the `MSE` tensor from Checkpoint 2 (hint: `variable**(1/2)` will compute the square root of a tensor assigned to `variable`). Assign the result to the variable `RMSE` (this stands for **root** mean squared error).
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
The code for finding the square root of `MSE` is `MSE**(1/2)`.

Setup
Run the following cell to import NumPy, pandas, and PyTorch.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
```

Checkpoint 1/3
Suppose we fed two apartments through a neural network with the result

- predictions: 750, 1000
- targets (actual rent): 1000, 900

We've tried to calculate MSE below, but we've made a mistake. Can you fix our calculation?
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
difference1 = 750 - 1000
difference2 = 1000 - 900
MSE = (difference1**2 + difference2**2)/2

# show output
MSE
```

```
36250.0
```

Checkpoint 2/3
In Exercise 7 we built the neural network

```
model = nn.Sequential(
    nn.Linear(3,16),
    nn.ReLU(),
    nn.Linear(16,8),
    nn.ReLU(),
    nn.Linear(8,4),
    nn.ReLU(),
    nn.Linear(4,1)
)
```

When we fed our apartment data forward through the network, the first five predictions were

```
predictions = torch.tensor([-6.9229, -29.8163, -16.0748, -13.2427, -14.1096], dtype=torch.float)
```

The first five targets (actual rent values) were

```
y = torch.tensor([2550, 11500, 3000, 4500, 4795], dtype=torch.float)
```

Use PyTorch's `nn.MSELoss()` function to compute the MSE of these five apartments. Assign the result to the variable `MSE`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# define prediction and target tensors
predictions = torch.tensor([-6.9229, -29.8163, -16.0748, -13.2427, -14.1096], dtype=torch.float)
y = torch.tensor([2550, 11500, 3000, 4500, 4795], dtype=torch.float)

## YOUR SOLUTION HERE ##
loss = nn.MSELoss()
MSE = loss(predictions, y)

# show output
print("MSE Loss:", MSE)
```

```
MSE Loss: tensor(38413624.)
```

Checkpoint 3/3
Let's make the result from Checkpoint 2 a bit more interpretable.
Compute the square root of the `MSE` tensor from Checkpoint 2 (hint: `variable**(1/2)` will compute the square root of a tensor assigned to `variable`). Assign the result to the variable `RMSE` (this stands for **root** mean squared error).
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
RMSE = MSE**(1/2)

# show output
RMSE
```

```
tensor(6197.8726)
```

Looks like this model's error for rent in dollars is around `$6200`. We can improve this quite a bit in training!

The Optimizer

With a loss function, we can tell how well (or poorly!) our neural network is performing. To improve on our model's performance, we need to adjust the weights and biases. This is what the **optimizer algorithm** does!

Gradient Descent Optimization

There are many different optimization algorithms. One of the most common is called **gradient descent**.

Imagine we're on top of a mountain, and our loss function tells us how high up we are. Our goal is to get down the mountain, making the loss function as small as possible. Unfortunately, it's a dark night, so even with perfect sight we couldn't see very far.

Suppose (using sight or some kind of radar) we can determine how the mountain slopes around us for about a meter in any direction. With only that information, we might pick where the mountain slopes down the most and move in that direction.

We don't want to move too far at once, because the mountain slope might change as we move. So we'll only move a short distance in our chosen direction before pausing and re-evaluating which direction goes downhill the most.

This strategy is essentially how gradient descent works! It uses calculus to determine the **gradients** of the loss function. These gradients are the direction signs that indicate which way to adjust the weights and biases in order to decrease the loss function.

Learning Rate

How far to move at each step is called the **learning rate**. Choosing a learning rate involves some tradeoffs:

- a learning rate *too high* may cause the model to move too quickly and miss the lowest point
- a learning rate *too small* may cause the model to learn slowly or get stuck

The learning rate is a classic example of what we call **hyperparameters** – values tuned and tweaked by ML engineers during training to improve performance of the model. The process of adjusting hyperparameters in search of the best model performance is called **hyperparameter tuning**.

Using Optimizers in PyTorch

A popular optimizer in PyTorch, called **Adam**, uses gradient descent with a few extra bells and whistles (like adjusting the learning rate dynamically during training). To use Adam, we'll use the syntax

```
import torch.optim as optim
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

Copy to Clipboard

where

- `model.parameters()` tells Adam what our current weights and biases are
- `lr=0.01` tells Adam to set the learning rate to `0.01`

To apply Adam to a neural network, we need to perform the:

- **backwards pass**: calculate the gradients of the loss function (these determine the "downward" direction)
- **step**: use the gradients to update the weights and biases

The syntax is

```
# compute the loss
MSE = loss(predictions,y)
# backward pass to determine "downward" direction
MSE.backward()
# apply the optimizer to update weights and biases
optimizer.step()
```

Copy to Clipboard

Note that `backward` is applied to the computed loss, not the loss function. This is why the output of the `loss` function includes the parameter `grad_fn=<MseLossBackward0>`. This parameter is the function used to perform the backwards pass.

Instructions

Checkpoint 1 Passed

1.

Import PyTorch's optimizers using the alias `optim`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Use the syntax

```
import library as alias
```

Copy to Clipboard

with `torch.optim` as the library.

Checkpoint 2 Passed

2.

We've defined a sequential neural network `model` already in the Checkpoint 2 code cell.

Initialize the Adam optimizer using `model`'s parameters and a **learning rate** set to `0.001`.

Save the result to the variable `optimizer`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Pass `model.parameters()` and `lr=.001` to the `optim.Adam` algorithm.

Checkpoint 3 Passed

3.

We've already defined a neural network, run a forward pass with some of our apartment data, and computed the loss from that forward pass.
At `## YOUR SOLUTION HERE ##`, add code to
- initialize `Adam` in the variable `optimizer` with a learning rate of `.001`
- use the loss we've already assigned to `MSE` to perform the backwards pass
- use `optimizer` to update the weights and biases

Run the notebook again. Did our optimization decrease the loss at all?
After testing your work with the learning rate of `.001`, try some other learning rates to see how that impacts the optimization.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
Apply `.backward()` to the variable containing the loss.
Then, apply `.step()` to the optimizer to update the weights and biases.

Import Libraries
Run the cell below to import NumPy, pandas, and PyTorch.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
```

Checkpoint 1/3
Import PyTorch's optimizers using the alias `optim`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
import torch.optim as optim
```

Checkpoint 2/3
We've defined a sequential neural network `model` already in the Checkpoint 2 code cell.
Initialize the Adam optimizer using `model`'s parameters and a **learning rate** set to `0.001`.
Save the result to the variable `optimizer`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

# create neural network
model = nn.Sequential(
    nn.Linear(3,16),
    nn.ReLU(),
    nn.Linear(16,8),
    nn.ReLU(),
    nn.Linear(8,4),
    nn.ReLU(),
    nn.Linear(4,1)
)

## YOUR SOLUTION HERE ##
optimizer = optim.Adam(model.parameters(),lr=0.001)
```

Checkpoint 3/3
We've already defined a neural network, run a forward pass with some of our apartment data, and computed the loss from that forward pass.
At `## YOUR SOLUTION HERE ##`, add code to

1. initialize `Adam` in the variable `optimizer` with a learning rate of `.001`
2. use the loss we've already assigned to `MSE` to perform the backwards pass
3. use `optimizer` to update the weights and biases

Run the notebook again. Did our optimization decrease the loss at all?
After testing your work with the learning rate of `.001`, try some other learning rates to see how that impacts the optimization.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

# create neural network
model = nn.Sequential(
    nn.Linear(3,16),
    nn.ReLU(),
    nn.Linear(16,8),
    nn.ReLU(),
    nn.Linear(8,4),
    nn.ReLU(),
    nn.Linear(4,1)
)

# import the data
apartments_df = pd.read_csv("streeteasy.csv")
numerical_features = ['bedrooms', 'bathrooms', 'size_sqft']
X = torch.tensor(apartments_df[numerical_features].values, dtype=torch.float)
y = torch.tensor(apartments_df['rent'].values,dtype=torch.float)

# forward pass
predictions = model(X)

# define the loss function and compute loss
loss = nn.MSELoss()
MSE = loss(predictions,y)
print('Initial loss is ' + str(MSE))

## YOUR SOLUTION HERE ##
optimizer = optim.Adam(model.parameters(), lr=0.001)
MSE.backward()
optimizer.step()

# feed the data through the updated model and compute the new loss
predictions = model(X)
MSE = loss(predictions,y)
print('After optimization, loss is ' + str(MSE))
```

```
Initial loss is tensor(29213900., grad_fn=<MseLossBackward0>)
After optimization, loss is tensor(29205546., grad_fn=<MseLossBackward0>)
```

Training

In the last exercise, we optimized weights and biases using the following code:

```
predictions = model(X) # forward pass
MSE = loss(predictions,y) # compute loss
MSE.backward() # compute gradients
optimizer.step() # update weights and biases
```

Copy to Clipboard

In this exercise, we'll turn this code into a **training loop** that will perform multiple optimizations in a row, attempting to decrease the loss as much as possible.

The only additional line of code we'll need inside the loop is `optimizer.zero_grad()`. This line resets the gradients for each iteration. The gradients determine the direction to move the weights and biases, and we want to pick a brand new direction each time.

Here's an example loop. Note that in neural networks, iterations of the training loop are called **epochs**.

```
num_epochs = 1000
for epoch in range(num_epochs):
    predictions = model(X) # forward pass
```

```
    MSE = loss(predictions,y) # compute loss
    MSE.backward() # compute gradients
    optimizer.step() # update weights and biases
    optimizer.zero_grad() # reset the gradients for the next iteration
```

Copy to Clipboard

We'll also want to print out the loss value regularly during training, to track the improvement of our model. Here's one example of how to print out the epoch number and loss every 100 epochs:

```
    # keep track of the loss values during training
    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{num_epochs}], MSE Loss: {MSE.item()}')
```

Copy to Clipboard

Note that:

since `epoch` starts at `0`, the 100th epoch occurs at `epoch = 99`. So the line `if (epoch + 1) % 100 == 0` checks if the current epoch is the 100th, 200th, and so on.

we call `.item()` on `MSE` to print out only the loss value. If we just called `MSE` on its own, we'd also print out the `grad_fn`.

Instructions

Checkpoint 1 Passed

1.

First, run the Setup and Import Data cells.

In the Checkpoint 1 code cell, we've created a neural network, defined a loss function, and initialized an optimizer.

Note on the network: since we now have `14` input features, we've increased the size of our hidden layers to try to capture that complexity. But we are still following a common neural network architecture where the first hidden layer has twice as many nodes (128) as the next (64).

We've also tried to write a training loop to train our neural network for 500 epochs, but we've made a mistake. As a result, our loss values are extremely high!

Can you spot our error and fix the training loop?

Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

The training loop needs to:

- run a forward pass
- compute the loss
- run a backward pass to compute gradients
- update the weights and biases
- reset the gradients so each iteration calculates them from scratch

Checkpoint 2 Passed

2.

Using the same model set up, we've written another training loop. This time we've made a different mistake, and the loss values don't seem to be changing from epoch to epoch.

Can you fix our mistake?

Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Since the loss isn't changing, something is going wrong in the optimization process that updates the weights and biases.

The optimizer is being called with `optimizer.step()`. What needs to happen before the weights and biases are updated?
Checkpoint 3 Passed

3.

We've already defined a neural network, loss function, and optimizer.
Write a training loop that will train the neural network `model` for 1000 epochs.
Do the additional epochs improve the model performance?
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
Try to fill in the loop from scratch, but if you are stuck, check our training loop code in the earlier checkpoints.

Setup

Run the following cell to import NumPy, pandas, and PyTorch.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
```

Import Data

Run the code cell to import the dataset and select numerical features. In this exercise, we'll start using a larger set of numerical features:

- **bedrooms**: The number of bedrooms in the apartment
- **bathrooms**: The number of bathrooms in the apartment
- **size_sqft**: The size of the apartment in square feet
- **min_to_subway**: The number of minutes to the closest subway
- **floor**: The building floor of the apartment
- **building_age_yrs**: The age of the building in years
- **no_fee**: Binary indicator that specifies whether the rental has a broker's fee (1) or not (0)
- **has_roofdeck**: Binary indicator that specifies whether the rental has a roofdeck (1) or not (0)
- **has_washer_dryer**: Binary indicator that specifies whether the rental has a washer/dryer units (1) or not (0)
- **has_doorman**: Binary indicator that specifies whether the rental has a doorman (1) or not (0)
- **has_elevator**: Binary indicator that specifies whether the rental has an elevator (1) or not (0)
- **has_dishwasher**: Binary indicator that specifies whether the rental has a dishwasher (1) or not (0)
- **has_patio**: Binary indicator that specifies whether the rental has a patio (1) or not (0)
- **has_gym**: Binary indicator that specifies whether the rental has a gym (1) or not (0) boroughs

```
apartments_df = pd.read_csv("streeteasy.csv")

numerical_features = ['bedrooms', 'bathrooms', 'size_sqft', 'min_to_subway', 'floor', 'building_age_yrs',
                      'no_fee', 'has_roofdeck', 'has_washer_dryer', 'has_doorman', 'has_elevator',
'has_dishwasher',
                      'has_patio', 'has_gym']

# create tensor of input features
X = torch.tensor(apartments_df[numerical_features].values, dtype=torch.float)
# create tensor of targets
y = torch.tensor(apartments_df['rent'].values, dtype=torch.float).view(-1,1)
```

Checkpoint 1/3

We've created a neural network, defined a loss function, and initialized an optimizer already.
Note on the network: since we now have `14` input features, we've increased the size of our hidden layers to try to capture that complexity. But we are still following a common neural network architecture where the first hidden layer has twice as many nodes (128) as the next (64).
We've also tried to write a training loop to train our neural network for 500 epochs, but we've made a mistake. As a result, our loss values are extremely high!
Can you spot our error and fix the training loop?
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)
```

```
# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(14, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)

# MSE loss function + optimizer
loss = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

## YOUR SOLUTION HERE ##
num_epochs = 500
for epoch in range(num_epochs):
    predictions = model(X)
    MSE = loss(predictions, y)
    MSE.backward()
    optimizer.step()
    optimizer.zero_grad()

    ## DO NOT MODIFY ##
    # keep track of the loss during training
    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{num_epochs}], MSE Loss: {MSE.item()}')
```

```
Epoch [100/500], MSE Loss: 3136162.75
Epoch [200/500], MSE Loss: 3019612.0
Epoch [300/500], MSE Loss: 2938490.0
Epoch [400/500], MSE Loss: 2867499.5
Epoch [500/500], MSE Loss: 2807188.5
```

Checkpoint 2/3

Using the same model set up, we've written another training loop. This time we've made a different mistake, and the loss values don't seem to be changing from epoch to epoch.

Can you fix our mistake?

Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(14, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)

# MSE loss function + optimizer
loss = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

## YOUR SOLUTION HERE ##
num_epochs = 500
for epoch in range(num_epochs):
    predictions = model(X)
    MSE = loss(predictions, y)
    MSE.backward()
    optimizer.step()
    optimizer.zero_grad()

    ## DO NOT MODIFY ##
    # keep track of the loss during training
    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{num_epochs}], MSE Loss: {MSE.item()}')
```

```
Epoch [100/500], MSE Loss: 3136162.75
Epoch [200/500], MSE Loss: 3019612.0
Epoch [300/500], MSE Loss: 2938490.0
Epoch [400/500], MSE Loss: 2867499.5
```

Epoch [500/500], MSE Loss: 2807188.5

Checkpoint 3/3
We've already defined a neural network, loss function, and optimizer.
Fill in a training loop that will train the neural network `model` for 1000 epochs.
Do the additional epochs improve the model performance?
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# set a random seed - do not modify
torch.manual_seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(14, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)

# MSE loss function + optimizer
loss = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

## YOUR SOLUTION HERE ##
num_epochs = 1000
for epoch in range(num_epochs):
    predictions = model(X) # forward pass
    MSE = loss(predictions, y) # calculate the loss
    MSE.backward()
    optimizer.step() # update the weights and biases
    optimizer.zero_grad()

    ## DO NOT MODIFY ##
    # keep track of the loss during training
    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{num_epochs}], MSE Loss: {MSE.item()}')
```

```
Epoch [100/1000], MSE Loss: 3136162.75
Epoch [200/1000], MSE Loss: 3019612.0
Epoch [300/1000], MSE Loss: 2938490.0
Epoch [400/1000], MSE Loss: 2867499.5
Epoch [500/1000], MSE Loss: 2807188.5
Epoch [600/1000], MSE Loss: 2757318.5
Epoch [700/1000], MSE Loss: 2716579.75
Epoch [800/1000], MSE Loss: 2683151.5
Epoch [900/1000], MSE Loss: 2656409.5
Epoch [1000/1000], MSE Loss: 2633397.0
```

Testing and Evaluation

In the last exercise, we ran training loops that decreased the loss from our neural network. But decreasing the loss during training only tells us that the model is becoming more accurate on the data it is learning from. We also need to

- evaluate the model on new data
- save the best model for future use

Evaluation
Instead of waiting to collect a new dataset, ML engineers usually only train models on a portion of the original dataset (called the **training data**) and then evaluate it on the remainder (called the **testing data**). This process is called the **train-test split**.
We'll use the Python library `scikit-learn` to perform the split. The syntax is

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
    train_size=0.80,
```

```
    test_size=0.20,
    random_state=2)
```

Copy to Clipboard

where

- `X` contains all of our input features
- `y` contains the targets we are trying to predict
- `train_size = .8` tells `scikit-learn` to use 80% of the data for training
- `test_size = .2` tells `scikit-learn` to use the remaining 20% for testing
- `random_state = 2` sets a random state, so that we create the same train-test split each time we run our code

This produces two sets of data:

- data for training the model: `X_train` (input features) and `y_train` (corresponding targets)
- data for testing the model: `X_test` (input features) and `y_test` (corresponding targets)

Evaluation

To evaluate our model on new data, we will use our testing data with the following syntax:

```
model.eval()
with torch.no_grad():
    predictions = model(X_test)
    test_MSE = loss(predictions, y_test)
```

Copy to Clipboard

where

- `model.eval()` sets the model to evaluation mode
- `with torch.no_grad()` turns off gradient calculations (which we don't need outside of training)

Otherwise, this code is basically the same as code we used in training:

- we run feedforward to predict the targets for the test dataset
- we compute the MSE loss between those predicted targets and the actual targets

Saving and Loading Models

Once we have an incredible rent-prediction model, we want to be able to save and load models without going through the entire training process again. The syntax is

```
# save the neural network to a specified path
torch.save(model, 'model.pth')

# load the saved neural network from the specified path
loaded_model = torch.load('model.pth')
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

In the Import Data cell, we created two tensors:

- `X` contains our numeric input features
- `y` contains the corresponding target rent values

Use `scikit-learn` to split these into training and testing datasets.
The training dataset (`X_train` and `y_train`) should have 70% of the data.
The testing dataset (`X_test` and `y_test`) should have 30% of the data.
Use a random state of `2`.
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Checkpoint 2 Passed

2.

We've already defined a neural network and training loop. But we forgot to only train on our training dataset! Can you fix the code so that the neural network is only seeing the training data during the training process?
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

The full dataset is

`x` for input features
`y` for targets

Wherever these are used in the training loop, we'll need to replace them with the corresponding training dataset variables.

Checkpoint 3 Passed

3.

Save the model from Checkpoint 2 to the file `model.pth`.

Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Use PyTorch's `model.save()` method with two inputs:

the model
the path to save it to as a string

Checkpoint 4 Passed

4.

We've saved a model trained for `20,000` epochs in the path `model20k.pth`.

Load this model to the variable `loaded_model`, and evaluate its performance on the test dataset.

Within the evaluation, save the MSE of the test predictions to the variable `test_MSE`.

Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Use PyTorch's `model.load()` method, passing the path to load as a string.

Then, evaluate by:

setting the model to evaluation mode
telling PyTorch not to compute gradients
passing the test data forward through the loaded model
computing the loss

Setup

Run the following cell to import NumPy, pandas, and PyTorch.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
```

Import Data

Run the code cell to import the dataset and select numerical features. In this exercise, we'll start using a larger set of numerical features:

- **bedrooms**: The number of bedrooms in the apartment
- **bathrooms**: The number of bathrooms in the apartment
- **size_sqft**: The size of the apartment in square feet
- **min_to_subway**: The number of minutes to the closest subway
- **floor**: The building floor of the apartment
- **building_age_yrs**: The age of the building in years
- **no_fee**: Binary indicator that specifies whether the rental has a broker's fee (1) or not (0)
- **has_roofdeck**: Binary indicator that specifies whether the rental has a roofdeck (1) or not (0)
- **has_washer_dryer**: Binary indicator that specifies whether the rental has a washer/dryer units (1) or not (0)
- **has_doorman**: Binary indicator that specifies whether the rental has a doorman (1) or not (0)
- **has_elevator**: Binary indicator that specifies whether the rental has an elevator (1) or not (0)
- **has_dishwasher**: Binary indicator that specifies whether the rental has a dishwasher (1) or not (0)
- **has_patio**: Binary indicator that specifies whether the rental has a patio (1) or not (0)
- **has_gym**: Binary indicator that specifies whether the rental has a gym (1) or not (0) boroughs

```
apartments_df = pd.read_csv("streeteasy.csv")

numerical_features = ['bedrooms', 'bathrooms', 'size_sqft', 'min_to_subway', 'floor', 'building_age_yrs',
                      'no_fee', 'has_roofdeck', 'has_washer_dryer', 'has_doorman', 'has_elevator',
'has_dishwasher',
                      'has_patio', 'has_gym']

# create tensor of input features
```

```
X = torch.tensor(apartments_df[numerical_features].values, dtype=torch.float)
# create tensor of targets
y = torch.tensor(apartments_df['rent'].values, dtype=torch.float).view(-1,1)
```

Checkpoint 1/4

In the Import Data cell, we created two tensors:

- `X` contains our numeric input features
- `y` contains the corresponding target rent values

Use `scikit-learn` to split these into training and testing datasets.
The training dataset (`X_train` and `y_train`) should have 70% of the data.
The testing dataset (`X_test` and `y_test`) should have 30% of the data.
Use a random state of `2`.
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
    train_size=0.70,
    test_size=0.30,
    random_state=2)
```

Checkpoint 2/4

We've already defined a neural network and training loop. But we forgot to only train on our training dataset! Can you fix the code so that the neural network is only seeing the training data during the training process?
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
torch.manual_seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(14, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)

# MSE loss function + optimizer
loss = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

## YOUR SOLUTION HERE ##
num_epochs = 1000
for epoch in range(num_epochs):
    predictions = model(X_train)
    MSE = loss(predictions, y_train)
    MSE.backward()
    optimizer.step()
    optimizer.zero_grad()

    ## DO NOT MODIFY ##
    # keep track of the loss during training
    if (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch + 1}/{num_epochs}], MSE Loss: {MSE.item()}')
```

```
Epoch [100/1000], MSE Loss: 3144756.75
Epoch [200/1000], MSE Loss: 3030343.5
Epoch [300/1000], MSE Loss: 2950239.0
Epoch [400/1000], MSE Loss: 2880079.0
Epoch [500/1000], MSE Loss: 2820468.25
Epoch [600/1000], MSE Loss: 2771156.5
Epoch [700/1000], MSE Loss: 2730910.0
Epoch [800/1000], MSE Loss: 2697787.75
Epoch [900/1000], MSE Loss: 2671344.25
Epoch [1000/1000], MSE Loss: 2648587.5
```

Checkpoint 3/4

Save the model from Checkpoint 2 to the file `model.pth`.
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
torch.save(model, 'model.pth')
```

Checkpoint 4/4

We've saved a model trained for `20,000` epochs in the path `model20k.pth`.

Load this model to the variable `loaded_model`, and evaluate its performance on the test dataset.
Within the evaluation, save the MSE of the test predictions to the variable `test_MSE`.
Don't forget to run and save the cell before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTIONS HERE ##
loaded_model = torch.load('model20k.pth')
loaded_model.eval()
with torch.no_grad():
   predictions = loaded_model(X_test)
   test_MSE = loss(predictions, y_test)

## DO NOT MODIFY ##
# show output
print('Test MSE is ' + str(test_MSE.item()))
print('Test Root MSE is ' + str(test_MSE.item()**(1/2)))
```

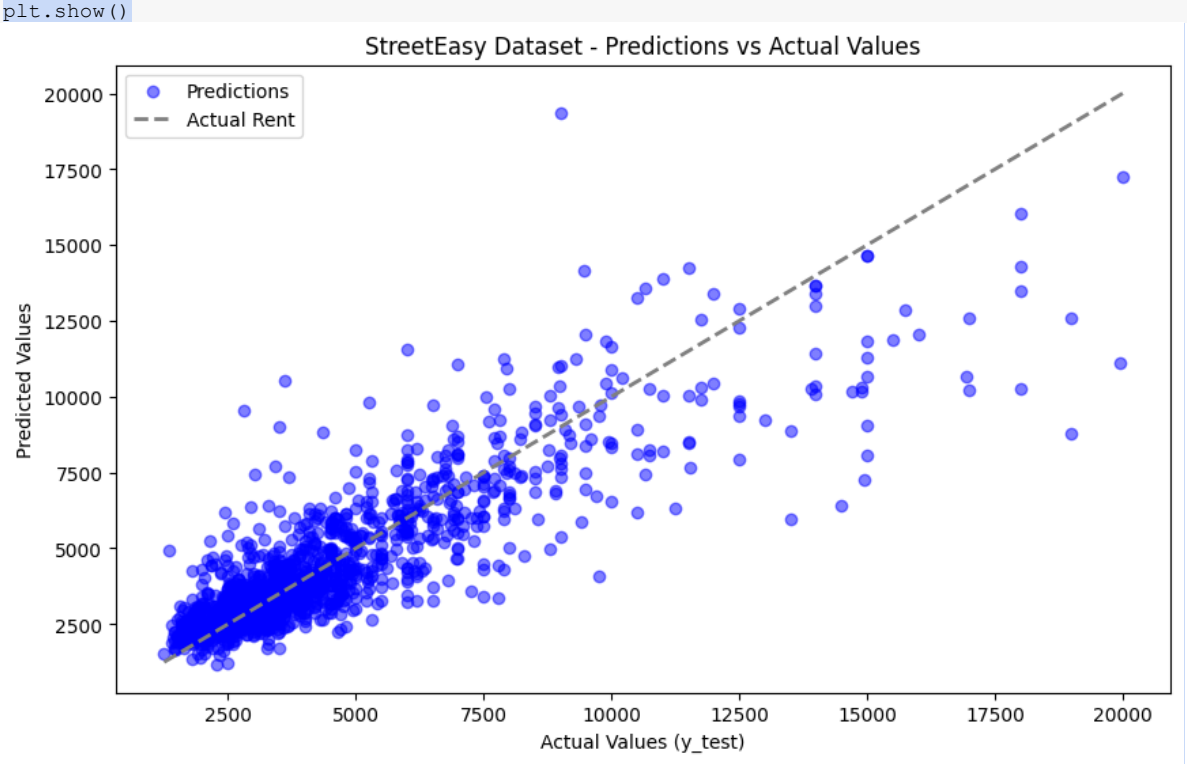
Test MSE is 1997976.875
Test Root MSE is 1413.4980986899134

On average, our model still has error probably larger than we'd want. But for a quite small neural network without any tuning, this is pretty good!

Visualization

Run the cell below to plot our model's predictions against the actual targets, for the test dataset. If our model was perfect, all the dots would be on the dashed line. How do you think we've done?

```
import matplotlib.pyplot as plt
%matplotlib inline

plt.figure(figsize=(10, 6))
plt.scatter(y_test, predictions, label='Predictions', alpha=0.5, color='blue')

plt.xlabel('Actual Values (y_test)')
plt.ylabel('Predicted Values')

plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--', color='gray',
linewidth=2,
         label="Actual Rent")
plt.legend()
plt.title('StreetEasy Dataset - Predictions vs Actual Values')
plt.show()
```



We're definitely moving in the right direction!

Review

Congratulations on finishing this lesson! You've taken a huge first step in your neural networks journey. Just in this lesson, you've built and trained a neural network from end to end. More specifically, you've

- turned linear equations into perceptron models
- learned about loss functions, activation functions, and optimizers
- built sequential neural networks
- built neural network classes using OOP
- trained and evaluated a neural network for predicting rent

We've created an open-ended Jupyter notebook for this exercise. Practice everything you've learned by building your own neural network from scratch, following the outline in the notebook. We won't be checking your work, this is a chance for you to practice and explore on your own. Many of the most important breakthroughs in neural networks came from experimentation!

Don't worry if you don't remember all the syntax from the prior exercises. It is completely normal to have to go back to earlier exercises to review the exact details. In fact, even experienced ML engineers often copy syntax from old projects to build new ones.

This is completely optional, and when you're ready to move on, select `Up Next`!

Next Steps

What's next?

Congratulations on completing Intro to PyTorch and Neural Networks! In this course, you learned about creating neural networks with PyTorch and are now able to:

- Build neural networks in PyTorch
- Define activation and loss functions
- Evaluate neural network performance
- Create real-world predictive models

Now that you have finished your course, you may be asking, "What's next?" Here are our recommendations for next steps:

PyTorch for Classification

Learn more about PyTorch! This course takes the next step in the neural network journey, covering *classification* models, where we try to predict categorical results instead of numeric results.

Intro to AI Transformers

Use PyTorch and Hugging Face's libraries to finetune transformers – the models that power ChatGPT

Learn Intermediate Python 3: Object-Oriented Programming

Learn object oriented programming to boost your understanding of OOP-based PyTorch neural networks.

Once again, congratulations on finishing your course! We're excited to see what you accomplish next.